```
import { useState, useEffect, useRef } from "react";

// ── DESIGN SYSTEM ──────────────────────────────────────────────
const D = {
  red:      "#ff1a1a",
  redDark:  "#cc0000",
  redDeep:  "#8b0000",
  redGlow:  "rgba(255,26,26,0.4)",
  redMid:   "rgba(255,26,26,0.15)",
  redSoft:  "rgba(255,26,26,0.06)",
  redBorder:"rgba(255,26,26,0.25)",

  bg:       "#050505",
  bgCard:   "rgba(255,255,255,0.025)",
  bgHover:  "rgba(255,255,255,0.05)",
  border:   "rgba(255,255,255,0.06)",
  borderHi: "rgba(255,26,26,0.3)",

  textPrimary:   "#ffffff",
  textSecondary: "#999999",
  textMuted:     "#555555",

  gold:   "#d4a017",
  goldSoft:"rgba(212,160,23,0.15)",
  green:  "#00e676",
  greenSoft:"rgba(0,230,118,0.12)",
  purple: "#9c27b0",

  shadowRed: "0 0 32px rgba(255,26,26,0.35)",
  shadowCard: "0 4px 24px rgba(0,0,0,0.6)",

  r:  8,
  rm: 12,
  rl: 16,
  rxl:20,

  heading: "'Bebas Neue', cursive",
  body:    "Georgia, serif",

  pad: "16px",
};

const GLOBAL_STYLES = `
  @import url('https://fonts.googleapis.com/css2?family=Bebas+Neue&display=swap')
```

```
  * { box-sizing: border-box; -webkit-tap-highlight-color: transparent; }
  ::-webkit-scrollbar { width: 0; background: transparent; }
  input, textarea, select { outline: none; }
  @keyframes fadeUp   { from{opacity:0;transform:translateY(14px)} to{opacity:1;t
  @keyframes fadeIn   { from{opacity:0} to{opacity:1} }
  @keyframes scaleIn  { from{opacity:0;transform:scale(0.94)} to{opacity:1;transf
  @keyframes slideDown{ from{opacity:0;transform:translateY(-10px)} to{opacity:1;
  @keyframes pulse    { 0%,100%{opacity:1} 50%{opacity:0.5} }
  @keyframes glow     { 0%,100%{filter:drop-shadow(0 0 8px rgba(255,26,26,0.5))}
  @keyframes spin     { from{transform:rotate(0deg)} to{transform:rotate(360deg)}
  @keyframes badgeIn  { from{opacity:0;transform:translateY(-24px) scale(0.9)} to
  @keyframes blink    { 0%,100%{opacity:1} 50%{opacity:0.3} }
  @keyframes shimmer  { 0%{background-position:-200% 0} 100%{background-position:
  @keyframes dot      { 0%,80%,100%{opacity:0.2;transform:scale(0.8)} 40%{opacity
  @keyframes celebrate{ 0%{transform:scale(1)} 30%{transform:scale(1.06)} 100%{tr
  @keyframes popIn    { from{transform:scale(0.7);opacity:0} to{transform:scale(1
  @keyframes flashFade{ from{opacity:1} to{opacity:0} }
  @keyframes splashRing{from{transform:scale(0.6);opacity:0} to{transform:scale(1
  @keyframes splashBar { from{width:0%} to{width:100%} }
  @keyframes rotateSlow{from{transform:rotate(0deg)} to{transform:rotate(360deg)}
  @keyframes splashGlow{0%,100%{filter:drop-shadow(0 0 12px rgba(255,26,26,0.4))}
`;

function Btn({ children, onClick, variant="primary", disabled, style={}, ...props
  const base = { border:"none", borderRadius:D.rm, padding:"13px 16px", cursor:di
  const variants = {
    primary:  { background: disabled ? `rgba(255,26,26,0.2)` : `linear-gradient(1
    secondary:{ background:"rgba(255,255,255,0.05)", color:D.textSecondary, borde
    gold:     { background:`linear-gradient(135deg,${D.gold},#b8860b)`, color:"#0
    ghost:    { background:"none", color:D.textMuted, border:`1px solid ${D.borde
    danger:   { background:"rgba(255,26,26,0.08)", color:D.red, border:`1px solid
  };
  return <button onClick={disabled?undefined:onClick} style={{...base,...variants
}

function Card({ children, style={}, accent, onClick }) {
  return (
    <div onClick={onClick} style={{ background:D.bgCard, border:`1px solid ${acce
      {children}
    </div>
  );
}

function SLabel({ children, color, style={} }) {
  return <div style={{ fontSize:10, letterSpacing:3, fontFamily:D.heading, color:
}
```

```
// ── SUPABASE CONFIG (actif sur Vercel) ──────────────────────
const SUPABASE_URL = "https://pnvyamjadclyzhrcnkbr.supabase.co";
const SUPABASE_KEY = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS

// Force auth locale partout sauf sur le vrai domaine Vercel
const IS_PRODUCTION = typeof window !== "undefined" &&
  (window.location.hostname === "autammuaythai.com" ||
   window.location.hostname === "www.autammuaythai.com" ||
   window.location.hostname.endsWith(".vercel.app"));

// ── AUTH (local persistant en preview, Supabase en production) ─
const supaAuth = {
  signUp: async ({ email, password, name }) => {
    if (IS_PRODUCTION) {
      // Vrai Supabase en production
      try {
        const r = await fetch(`${SUPABASE_URL}/auth/v1/signup`, {
          method: "POST",
          headers: { "apikey": SUPABASE_KEY, "Content-Type": "application/json" }
          body: JSON.stringify({ email, password }),
        });
        const data = await r.json();
        if (data.error) return { error: data.error.message || "Erreur inscription
        const userId = data.user?.id;
        const token = data.access_token;
        await fetch(`${SUPABASE_URL}/rest/v1/profiles`, {
          method: "POST",
          headers: { "apikey": SUPABASE_KEY, "Authorization": `Bearer ${token}`,
          body: JSON.stringify({ id: userId, email, name, created_at: new Date().
        });
        const user = { id: userId, email, name, createdAt: Date.now(), token };
        localStorage.setItem("amt_session", JSON.stringify(user));
        return { user, error: null };
      } catch { return { error: "Connexion impossible." }; }
    } else {
      // Auth locale persistante pour la prévisualisation
      const users = JSON.parse(localStorage.getItem("amt_users") || "{}");
      if (users[email]) return { error: "Cet email est déjà utilisé." };
      const user = { id: Date.now().toString(), email, name, password, createdAt:
      users[email] = user;
      localStorage.setItem("amt_users", JSON.stringify(users));
      localStorage.setItem("amt_session", JSON.stringify(user));
      return { user, error: null };
    }
  },

  signIn: async ({ email, password }) => {
```

```
  if (IS_PRODUCTION) {
    try {
      const r = await fetch(`${SUPABASE_URL}/auth/v1/token?grant_type=password`
        method: "POST",
        headers: { "apikey": SUPABASE_KEY, "Content-Type": "application/json" }
        body: JSON.stringify({ email, password }),
      });
      const data = await r.json();
      if (data.error_description) return { error: "Email ou mot de passe incorr
      const userId = data.user?.id;
      const token = data.access_token;
      const pr = await fetch(`${SUPABASE_URL}/rest/v1/profiles?id=eq.${userId}&
        headers: { "apikey": SUPABASE_KEY, "Authorization": `Bearer ${token}` }
      });
      const profiles = await pr.json();
      const profile = profiles[0];
      const user = { id: userId, email, name: profile?.name || email.split("@")
      localStorage.setItem("amt_session", JSON.stringify(user));
      return { user, error: null };
    } catch { return { error: "Connexion impossible." }; }
  } else {
    // Auth locale persistante
    const users = JSON.parse(localStorage.getItem("amt_users") || "{}");
    const user = users[email];
    if (!user) return { error: "Aucun compte trouvé. Inscris-toi d'abord." };
    if (user.password !== password) return { error: "Mot de passe incorrect." }
    localStorage.setItem("amt_session", JSON.stringify(user));
    return { user, error: null };
  }
},

signOut: () => {
  localStorage.removeItem("amt_session");
  localStorage.removeItem("amt_token");
},

getSession: () => {
  try {
    const s = localStorage.getItem("amt_session");
    return s ? JSON.parse(s) : null;
  } catch { return null; }
},

getTrialDaysLeft: (user) => {
  if (!user?.createdAt) return 7;
  const days = Math.floor((Date.now() - user.createdAt) / (1000 * 60 * 60 * 24)
  return Math.max(0, 7 - days);
```

```
  },
};


// ── AUTH SCREEN ──────────────────────────────────────────────

// ── DATA STORAGE (must be defined early) ────────────────────
const ALL_BADGES = [
  // Séances
  { id:"s1", icon:"🥊", name:"Premier Pas", desc:"Complète ta 1ère séance", categ
  { id:"s5", icon:"💪", name:"Guerrier", desc:"5 séances complétées", category:"S
  { id:"s10", icon:"🔥", name:"Combattant", desc:"10 séances complétées", categor
  { id:"s25", icon:"⚡", name:"Champion", desc:"25 séances complétées", category:
  { id:"s50", icon:"🏆", name:"Légende", desc:"50 séances complétées", category:"
  // Streak
  { id:"st3", icon:"🌟", name:"3 Jours de feu", desc:"3 jours de streak", categor
  { id:"st7", icon:"🔥", name:"Semaine parfaite", desc:"7 jours de streak", categ
  { id:"st14", icon:"💫", name:"Deux semaines", desc:"14 jours de streak", categc
  { id:"st30", icon:"👑", name:"Mois de gloire", desc:"30 jours de streak", categ
  // Coach IA
  { id:"ai1", icon:"🤖", name:"Premier Élève", desc:"1ère analyse Coach IA", cate
  { id:"ai5", icon:"🎯", name:"Apprenti", desc:"5 analyses Coach IA", category:"C
  { id:"ai10", icon:"🧠", name:"Technicien", desc:"10 analyses Coach IA", categor
  { id:"ai25", icon:"🎓", name:"Expert", desc:"25 analyses Coach IA", category:"C
  // Techniques
  { id:"t1", icon:"👊", name:"Première Frappe", desc:"1ère technique consultée",
  { id:"t3", icon:"🦵", name:"En mouvement", desc:"3 techniques maîtrisées", cate
  { id:"t6", icon:"💥", name:"Arsenal complet", desc:"Toutes les techniques vues"
];

const badgeStorage = {
  getStats: () => {
    try { return JSON.parse(localStorage.getItem("amt_badge_stats") || '{"session
    catch { return {sessions:0,streak:0,analyses:0,techniques:0}; }
  },
  setStats: (s) => localStorage.setItem("amt_badge_stats", JSON.stringify(s)),
  getUnlocked: () => {
    try { return JSON.parse(localStorage.getItem("amt_badges_unlocked") || "[]");
    catch { return []; }
  },
  unlock: (id) => {
    const u = badgeStorage.getUnlocked();
    if(!u.includes(id)) { u.push(id); localStorage.setItem("amt_badges_unlocked",
    return false;
  },
  checkAndUnlock: (stats) => {
    const newBadges = [];
```

```
    ALL_BADGES.forEach(b => {
      if(b.condition(stats) && badgeStorage.unlock(b.id)) newBadges.push(b);
    });
    return newBadges;
  },
};

const progression = {
  getStats: () => {
    try {
      return JSON.parse(localStorage.getItem("amt_prog_stats") || JSON.stringify(
        sessions: 0, totalMinutes: 0, totalCalories: 0,
        streak: 0, lastSessionDate: null,
        weeklySessionss: [0,0,0,0,0,0,0],
        history: [],
      }));
    } catch { return {sessions:0,totalMinutes:0,totalCalories:0,streak:0,lastSess
  },
  saveStats: (s) => localStorage.setItem("amt_prog_stats", JSON.stringify(s)),
  addSession: (minutes, type="Entraînement") => {
    const s = progression.getStats();
    const today = new Date().toDateString();
    const cal = Math.round(minutes * 8.5);
    if(s.lastSessionDate === today) {
      // Already trained today
    } else if(s.lastSessionDate === new Date(Date.now()-864e5).toDateString()) {
      s.streak += 1;
    } else {
      s.streak = 1;
    }
    s.sessions += 1;
    s.totalMinutes += minutes;
    s.totalCalories += cal;
    s.lastSessionDate = today;
    const day = new Date().getDay();
    if(!s.weeklySessionss) s.weeklySessionss = [0,0,0,0,0,0,0];
    s.weeklySessionss[day] = (s.weeklySessionss[day]||0) + 1;
    if(!s.history) s.history = [];
    s.history.unshift({date: new Date().toLocaleDateString("fr-FR"), minutes, cal
    if(s.history.length > 50) s.history = s.history.slice(0,50);
    progression.saveStats(s);
    badgeStorage.setStats({sessions:s.sessions, streak:s.streak, analyses:badgeSt
    return {s, cal, newBadges: badgeStorage.checkAndUnlock({sessions:s.sessions,
  },
};

function AuthScreen({ onAuth, onShowLegal }) {
```

```
const [mode, setMode] = useState("login");
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [name, setName] = useState("");
const [acceptedCGU, setAcceptedCGU] = useState(false);
const [error, setError] = useState("");
const [success, setSuccess] = useState("");
const [loading, setLoading] = useState(false);

const S = {
  input: {
    width: "100%", background: "rgba(255,255,255,0.07)",
    border: "1px solid rgba(255,255,255,0.12)", borderRadius: 12,
    color: "#fff", fontSize: 14, padding: "13px 16px",
    boxSizing: "border-box", fontFamily: "Georgia, serif",
    outline: "none",
  },
  label: { fontSize: 11, color: "#888", letterSpacing: 1, marginBottom: 6, disp
};

const handleSubmit = async () => {
  setError(""); setSuccess("");
  if (!email || !password) { setError("Remplis tous les champs."); return; }
  if (password.length < 6) { setError("Mot de passe : 6 caractères minimum.");
  setLoading(true);

  if (mode === "register") {
    if (!name.trim()) { setError("Entre ton prénom et nom."); setLoading(false)
    if (!acceptedCGU) { setError("Tu dois accepter les CGU et la politique de c
    const { user, error } = await supaAuth.signUp({ email, password, name });
    if (error) { setError(error); setLoading(false); return; }
    onAuth(user);
  } else {
    const { user, error } = await supaAuth.signIn({ email, password });
    if (error) { setError(error); setLoading(false); return; }
    onAuth(user);
  }
  setLoading(false);
};

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    {/* Header */}
    <div style={{background: "linear-gradient(135deg, #0a0a0a, #1a0000)", paddi
      <div style={{ fontSize: 52, marginBottom: 10, filter: "drop-shadow(0 0 12
      <div style={{ fontSize: 36, fontFamily: "'Bebas Neue', cursive", color: "
      <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color: "
```

```
  <div style={{ fontSize: 11, color: "#666", marginTop: 6, fontStyle: "ital
</div>

{/* Form */}
<div style={{ flex: 1, overflowY: "auto", padding: "24px 20px" }}>
  {/* Tab switcher */}
  <div style={{ display: "flex", background: "rgba(255,255,255,0.05)", bord
    {["login", "register"].map(m => (
      <button key={m} onClick={() => { setMode(m); setError(""); setSuccess
        {m === "login" ? "CONNEXION" : "INSCRIPTION"}
      </button>
    ))}
  </div>

  {/* Trial badge sur inscription */}
  {mode === "register" && (
    <div style={{background: "rgba(245,158,11,0.1)", border: "1px solid rgb
      <span style={{ fontSize: 20 }}>🎁</span>
      <div>
        <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", co
          7 JOURS PREMIUM OFFERTS
        </div>
        <div style={{ fontSize: 10, color: "#888" }}>Sans carte bancaire ·
      </div>
    </div>
  )}

  <div style={{ display: "flex", flexDirection: "column", gap: 16 }}>
    {mode === "register" && (
      <div>
        <span style={S.label}>PRÉNOM ET NOM</span>
        <input placeholder="Ex: Mathis Pozner" value={name} onChange={e =>
          style={S.input} />
      </div>
    )}
    <div>
      <span style={S.label}>EMAIL</span>
      <input type="email" placeholder="ton@email.com" value={email} onChang
        style={S.input} autoCapitalize="none" />
    </div>
    <div>
      <span style={S.label}>MOT DE PASSE</span>
      <input type="password" placeholder="6 caractères minimum" value={pass
        style={S.input} />
    </div>
  </div>
```

```
    {/* CGU checkbox — inscription seulement */}
    {mode === "register" && (
      <div onClick={() => setAcceptedCGU(a => !a)} style={{display: "flex", a
        <div style={{width: 22, height: 22, borderRadius: 6, flexShrink: 0, b
        <div style={{ fontSize: 11, color: "#aaa", lineHeight: 1.6 }}>
          J'ai lu et j'accepte les{" "}
          <span onClick={e => { e.stopPropagation(); onShowLegal("terms"); }}
            Conditions Générales d'Utilisation
          </span>{" "}et la{" "}
          <span onClick={e => { e.stopPropagation(); onShowLegal("privacy");
            Politique de Confidentialité
          </span>{" "}d'Autam Muay Thai.
        </div>
      </div>
    )}

    {/* Error / Success */}
    {error && (
      <div style={{ background: "rgba(239,68,68,0.1)", border: "1px solid rgb
        ⚠ {error}
      </div>
    )}
    {success && (
      <div style={{ background: "rgba(34,197,94,0.1)", border: "1px solid rgb
        {success}
      </div>
    )}

    {/* Submit */}
    <button onClick={handleSubmit} disabled={loading} style={{width: "100%",
      {loading ? "⏳ CHARGEMENT..." : mode === "login" ? "🥊 SE CONNECTER" :
    </button>

    {/* Google */}
    <div style={{ position: "relative", margin: "20px 0", textAlign: "center"
      <div style={{ height: 1, background: "rgba(255,255,255,0.08)" }} />
      <span style={{ position: "absolute", top: -8, left: "50%", transform: "
    </div>

    <button onClick={() => setError("Connexion Google disponible dans la vers
      <span style={{ fontSize: 18 }}>G</span> CONTINUER AVEC GOOGLE
    </button>

    <div style={{ textAlign: "center", marginTop: 16, fontSize: 10, color: "#
      En créant un compte tu acceptes nos CGU<br/>et notre politique de confi
    </div>
  </div>
```

```
    </div>
  );
}

const combos = [
  { id: 1, name: "Combo Débutant", level: "Débutant", strikes: ["Jab", "Cross", "
  { id: 2, name: "Combo Chasseur", level: "Intermédiaire", strikes: ["Jab", "Cros
  { id: 3, name: "Combo Guerrier", level: "Avancé", strikes: ["Jab", "Cross", "Co
];

const techniques = [
  { id: 1, name: "Teep", category: "Coup de pied", emoji: "🦵", desc: "Coup de pi
  { id: 2, name: "Roundhouse Kick", category: "Coup de pied", emoji: "🦵", desc:
  { id: 3, name: "Coude Horizontal", category: "Coude", emoji: "💥", desc: "Frapp
  { id: 4, name: "Genou Sauté", category: "Genou", emoji: "🔥", desc: "Genou expl
  { id: 5, name: "Clinch", category: "Corps à corps", emoji: "🤼", desc: "Positic
  { id: 6, name: "Jab-Cross", category: "Poing", emoji: "👊", desc: "Combinaison
];

const programs = [
  { id: 1, name: "Fondamentaux", weeks: 4, level: "Débutant", sessions: 3, color:
  { id: 2, name: "Combat Conditioning", weeks: 6, level: "Intermédiaire", session
  { id: 3, name: "Guerrier Élite", weeks: 8, level: "Avancé", sessions: 5, color:
  { id: 4, name: "Mon Programme", weeks: "∞", level: "Personnalisé", sessions: "?
];

// ── COACH IA SCREEN ──────────────────────────────────────────────
const DEFAULT_COMBOS = [
  {
    id: "c1", name: "Jab / Cross / Right Middle Kick",
    steps: ["Jab (poing avant)", "Cross (poing arrière)", "Right Middle Kick (kic
    cues: ["Garde haute pendant toute la séquence", "Pivot du pied arrière sur le
    refVideoUrl: null,
  },
  {
    id: "c2", name: "Jab / Cross / Low Kick",
    steps: ["Jab (poing avant)", "Cross (poing arrière)", "Low Kick (kick bas sur
    cues: ["Enchaîner rapidement jab-cross", "Frapper avec le tibia, pas le pied"
    refVideoUrl: null,
  },
  {
    id: "c3", name: "Jab / Cross / Crochet / Genou",
    steps: ["Jab", "Cross", "Crochet gauche", "Genou droit"],
    cues: ["Rotation du buste sur le crochet", "Saisir la nuque sur le genou (cli
    refVideoUrl: null,
  },
];
```

```
function CoachIAScreen({ isPremium, dailyAnalyses, setDailyAnalyses, goPaywall })
  const [tab, setTab] = useState("analyse"); // analyse | combos | historique
  const [mode, setMode] = useState("menu");  // menu | video | live
  const [customCombos, setCustomCombos] = useState(DEFAULT_COMBOS);
  const [selectedComboId, setSelectedComboId] = useState("c1");
  const [videoFile, setVideoFile] = useState(null);
  const [videoPreview, setVideoPreview] = useState(null);
  const [coachNotes, setCoachNotes] = useState("");
  const [analyzing, setAnalyzing] = useState(false);
  const [analysis, setAnalysis] = useState(null);
  const [history, setHistory] = useState([]);
  const [liveActive, setLiveActive] = useState(false);
  const [liveAnalysis, setLiveAnalysis] = useState(null);
  const [liveAnalyzing, setLiveAnalyzing] = useState(false);
  const [showDetailHistory, setShowDetailHistory] = useState(null);

  const [showCreateCombo, setShowCreateCombo] = useState(false);
  const [newComboName, setNewComboName] = useState("");
  const [newComboSteps, setNewComboSteps] = useState("");
  const [newComboCues, setNewComboCues] = useState("");
  const [refVideoFile, setRefVideoFile] = useState(null);
  const [refVideoPreview, setRefVideoPreview] = useState(null);
  const [editingComboId, setEditingComboId] = useState(null);

  const fileInputRef = useRef(null);
  const refVideoInputRef = useRef(null);
  const videoRef = useRef(null);
  const streamRef = useRef(null);
  const canvasRef = useRef(null);

  const selectedCombo = customCombos.find(c => c.id === selectedComboId) || custo

  // ── Helpers ──
  const toBase64 = (file) => new Promise((res, rej) => {
    const r = new FileReader();
    r.onload = () => res(r.result.split(",")[1]);
    r.onerror = rej;
    r.readAsDataURL(file);
  });

  const extractFrame = (file) => new Promise((resolve) => {
    const vid = document.createElement("video");
    vid.src = URL.createObjectURL(file);
    vid.onloadeddata = () => {
      const canvas = document.createElement("canvas");
      canvas.width = vid.videoWidth || 640;
```

```
    canvas.height = vid.videoHeight || 480;
    canvas.getContext("2d").drawImage(vid, 0, 0);
    resolve(canvas.toDataURL("image/jpeg").split(",")[1]);
  };
  vid.load();
});

// ── Créer / modifier combo ──
const saveCombo = () => {
  if (!newComboName.trim()) return;
  const steps = newComboSteps.split("\n").map(s => s.trim()).filter(Boolean);
  const cues = newComboCues.split("\n").map(s => s.trim()).filter(Boolean);
  if (editingComboId) {
    setCustomCombos(prev => prev.map(c => c.id === editingComboId
      ? { ...c, name: newComboName, steps, cues, refVideoUrl: refVideoPreview |
      : c));
  } else {
    const newId = "c" + Date.now();
    setCustomCombos(prev => [...prev, { id: newId, name: newComboName, steps, c
    setSelectedComboId(newId);
  }
  setShowCreateCombo(false);
  setEditingComboId(null);
  setNewComboName(""); setNewComboSteps(""); setNewComboCues("");
  setRefVideoFile(null); setRefVideoPreview(null);
};

const startEdit = (combo) => {
  setEditingComboId(combo.id);
  setNewComboName(combo.name);
  setNewComboSteps(combo.steps.join("\n"));
  setNewComboCues(combo.cues.join("\n"));
  setRefVideoPreview(combo.refVideoUrl || null);
  setShowCreateCombo(true);
};

const deleteCombo = (id) => {
  setCustomCombos(prev => prev.filter(c => c.id !== id));
  if (selectedComboId === id) setSelectedComboId(customCombos[0]?.id);
};

// ── Analyse vidéo ──
const analyzeVideo = async () => {
  if (!videoFile) return;
  if (!isPremium && dailyAnalyses >= 1) {
    goPaywall(); return;
  }
```

```
  setAnalyzing(true); setAnalysis(null);
  try {
    let imageData, mediaType = "image/jpeg";
    if (videoFile.type.startsWith("video/")) {
      imageData = await extractFrame(videoFile);
    } else {
      imageData = await toBase64(videoFile);
      mediaType = videoFile.type;
    }
    const refNote = selectedCombo.refVideoUrl ? "Le coach a fourni une vidéo de
    const response = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514",
        max_tokens: 1000,
        system: `Tu es un coach expert en Muay Thai. Analyse la technique visib
        messages: [{
          role: "user", content: [
            { type: "image", source: { type: "base64", media_type: mediaType, d
            { type: "text", text: `Combo: ${selectedCombo.name}\nSéquence: ${se
          ]
        }]
      })
    });
    const data = await response.json();
    const text = data.content?.map(c => c.text || "").join("") || "";
    const parsed = JSON.parse(text.replace(/```json|```/g, "").trim());
    setAnalysis(parsed);
    setDailyAnalyses(n => n + 1);
    setHistory(prev => [{
      id: Date.now(), date: new Date().toLocaleDateString("fr-FR"),
      comboName: selectedCombo.name, score: parsed.score,
      resume: parsed.resume, corrections: parsed.corrections,
      points_positifs: parsed.points_positifs, conseil_global: parsed.conseil_g
      imagePreview: videoPreview,
    }, ...prev]);
  } catch {
    setAnalysis({ error: "Erreur d'analyse. Essaie avec une image nette." });
  }
  setAnalyzing(false);
};

// ── Live ──
const startLive = async () => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ video: { facingM
```

```
    streamRef.current = stream;
    if (videoRef.current) videoRef.current.srcObject = stream;
    setLiveActive(true);
  } catch { alert("Impossible d'accéder à la caméra."); }
};
const stopLive = () => {
  streamRef.current?.getTracks().forEach(t => t.stop());
  setLiveActive(false); setLiveAnalysis(null);
};
const captureAndAnalyzeLive = async () => {
  if (!videoRef.current || liveAnalyzing) return;
  setLiveAnalyzing(true);
  try {
    const canvas = canvasRef.current;
    canvas.width = videoRef.current.videoWidth || 320;
    canvas.height = videoRef.current.videoHeight || 240;
    canvas.getContext("2d").drawImage(videoRef.current, 0, 0);
    const imageData = canvas.toDataURL("image/jpeg", 0.7).split(",")[1];
    const response = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514", max_tokens: 400,
        system: `Coach Muay Thai temps réel. JSON uniquement: {"posture":"bon/c
        messages: [{ role: "user", content: [
          { type: "image", source: { type: "base64", media_type: "image/jpeg",
          { type: "text", text: `Combo: ${selectedCombo.name}. Correction insta
        ]}]
      })
    });
    const data = await response.json();
    const text = data.content?.map(c => c.text || "").join("") || "";
    setLiveAnalysis(JSON.parse(text.replace(/```json|```/g, "").trim()));
  } catch {}
  setLiveAnalyzing(false);
};

// ── Styles communs ──
const S = {
  card: { background: "rgba(255,255,255,0.03)", border: "1px solid rgba(255,255
  label: { fontSize: 11, letterSpacing: 3, fontFamily: "'Bebas Neue', cursive",
  input: { width: "100%", background: "rgba(255,255,255,0.05)", border: "1px so
  btn: (color) => ({ width: "100%", background: color, border: "none", color: "
  back: { background: "rgba(255,255,255,0.06)", border: "1px solid rgba(255,255
};

if (tab === "combos") return (
  <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto", p
```

```
{!isPremium && <PremiumLock feature="Crée tes propres combos, ajoute des vi
<div style={{ display: "flex", justifyContent: "space-between", alignItems:
  <div>
    <div style={{ ...S.label, color: "#a855f7", marginBottom: 2 }}>MES COMB
    <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", color:
  </div>
  <button onClick={() => { setShowCreateCombo(true); setEditingComboId(null
</div>

{/* Formulaire création/édition */}
{showCreateCombo && (
  <div style={{ ...S.card, border: "1px solid rgba(168,85,247,0.4)", margin
    <div style={{ ...S.label, color: "#a855f7" }}>{editingComboId ? "MODIFI
    <input placeholder="Nom du combo (ex: Jab / Cross / Teep)" value={newCo
      style={{ ...S.input, marginBottom: 10 }} />
    <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}>Frappes (
    <textarea placeholder={"Jab\nCross\nRight Middle Kick"} value={newCombo
      style={{ ...S.input, resize: "none", height: 80, marginBottom: 10 }}
    <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}>Points cl
    <textarea placeholder={"Garde haute\nRotation des hanches\nRetour en ga
      style={{ ...S.input, resize: "none", height: 80, marginBottom: 12 }}

    {/* Vidéo de référence du coach */}
    <div style={{ fontSize: 10, color: "#f59e0b", marginBottom: 6, fontFami
    <div onClick={() => refVideoInputRef.current?.click()} style={{border:
      <input ref={refVideoInputRef} type="file" accept="video/*,image/*" st
        const f = e.target.files[0]; if (!f) return;
        setRefVideoFile(f); setRefVideoPreview(URL.createObjectURL(f));
      }} />
      {refVideoPreview ? (
        refVideoFile?.type?.startsWith("video") || !refVideoFile ?
          <video src={refVideoPreview} controls style={{ width: "100%", bor
          <img src={refVideoPreview} alt="ref" style={{ width: "100%", bord
      ) : (
        <div style={{ fontSize: 12, color: "#888" }}>📁 Ajouter ta vidéo de
      )}
    </div>

    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 }
      <button onClick={() => setShowCreateCombo(false)} style={{ ...S.btn("
      <button onClick={saveCombo} style={{ ...S.btn("#a855f7") }}>ENREGISTR
    </div>
  </div>
)}

{/* Liste des combos */}
<div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
```

```
      {customCombos.map(c => (
        <div key={c.id} style={{ ...S.card, border: selectedComboId === c.id ?
          <div style={{ display: "flex", justifyContent: "space-between", align
            <div style={{ flex: 1 }}>
              <div style={{ fontSize: 14, fontFamily: "'Bebas Neue', cursive",
              <div style={{ fontSize: 10, color: "#666", marginTop: 3 }}>{c.ste
            </div>
            <div style={{ display: "flex", gap: 6, marginLeft: 8 }}>
              <button onClick={() => startEdit(c)} style={{ background: "rgba(2
              {c.id.startsWith("c") && !["c1","c2","c3"].includes(c.id) && (
                <button onClick={() => deleteCombo(c.id)} style={{ background:
              )}
            </div>
          </div>
          {c.refVideoUrl && (
            <div style={{ fontSize: 10, color: "#f59e0b", marginBottom: 8 }}>📹
          )}
          <button onClick={() => { setSelectedComboId(c.id); setTab("analyse");
            {selectedComboId === c.id ? "✓ SÉLECTIONNÉ" : "SÉLECTIONNER"}
          </button>
        </div>
      ))}
    </div>
  </div>
);

if (tab === "historique") {
  if (showDetailHistory) {
    const h = showDetailHistory;
    return (
      <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto
        <div style={{ display: "flex", alignItems: "center", gap: 10, marginBot
          <button onClick={() => setShowDetailHistory(null)} style={S.back}>←</
          <div style={{ fontSize: 16, fontFamily: "'Bebas Neue', cursive", colo
        </div>
        <div style={{ fontSize: 11, color: "#888", marginBottom: 16 }}>{h.date}
        <div style={{ ...S.card, textAlign: "center", marginBottom: 12, backgro
          <div style={{ fontSize: 48, fontFamily: "'Bebas Neue', cursive", colo
          <div style={{ fontSize: 12, color: "#ccc", fontStyle: "italic", margi
        </div>
        {h.points_positifs?.length > 0 && (
          <div style={{ ...S.card, background: "rgba(34,197,94,0.07)", border:
            <div style={{ ...S.label, color: "#22c55e" }}>✅ POINTS POSITIFS</d
            {h.points_positifs.map((p, i) => <div key={i} style={{ fontSize: 12
          </div>
        )}
        {h.corrections?.length > 0 && (
```

```
        <div style={{ ...S.card, background: "rgba(239,68,68,0.07)", border:
          <div style={{ ...S.label, color: "#ef4444" }}>🔧 CORRECTIONS</div>
          {h.corrections.map((c, i) => (
            <div key={i} style={{ marginBottom: 10 }}>
              <div style={{ fontSize: 12, color: "#ef4444", fontFamily: "'Beb
              <div style={{ fontSize: 11, color: "#888" }}>⚠ {c.probleme}</di
              <div style={{ fontSize: 11, color: "#ccc" }}>→ {c.correction}</
            </div>
          ))}
        </div>
      )}
      {h.conseil_global && (
        <div style={{ ...S.card, background: "rgba(168,85,247,0.07)", border:
          <div style={{ ...S.label, color: "#a855f7" }}>💡 CONSEIL</div>
          <div style={{ fontSize: 12, color: "#ccc", fontStyle: "italic" }}>{
        </div>
      )}
    </div>
  );
}

return (
  <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto",
    {!isPremium && <PremiumLock feature="L'historique complet et les graphiqu
    <div style={{ ...S.label, color: "#22c55e", marginBottom: 2 }}>HISTORIQUE
    <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", color: "

    {history.length === 0 ? (
      <div style={{ ...S.card, textAlign: "center", padding: 32 }}>
        <div style={{ fontSize: 36, marginBottom: 12 }}>📊</div>
        <div style={{ fontSize: 13, color: "#666" }}>Aucune analyse pour l'in
        <div style={{ fontSize: 11, color: "#555", marginTop: 6 }}>Lance une
      </div>
    ) : (
      <>
        {/* Graphique d'évolution des scores */}
        {history.length > 1 && (
          <div style={{ ...S.card, marginBottom: 16 }}>
            <div style={{ ...S.label, color: "#a855f7" }}>ÉVOLUTION DU SCORE<
            <div style={{ display: "flex", alignItems: "flex-end", gap: 6, he
              {[...history].reverse().map((h, i) => (
                <div key={h.id} style={{ flex: 1, display: "flex", flexDirect
                  <div style={{width: "100%", borderRadius: 4, height: `${(h.
                  <span style={{ fontSize: 8, color: "#555" }}>{h.score}</spa
                </div>
              ))}
            </div>
```

```
            </div>
          )}

          {/* Liste analyses */}
          <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
            {history.map(h => (
              <div key={h.id} onClick={() => setShowDetailHistory(h)} style={{
                <div style={{width: 52, height: 52, borderRadius: 12, flexShrin
                <div style={{ flex: 1 }}>
                  <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursiv
                  <div style={{ fontSize: 11, color: "#666", marginTop: 2 }}>{h
                  <div style={{ fontSize: 11, color: "#888", marginTop: 2, font
                </div>
                <span style={{ color: "#555", fontSize: 14 }}>›</span>
              </div>
            ))}
          </div>
        </>
      )}
    </div>
  );
}

if (mode === "live") return (
  <div style={{ padding: "0 0 80px", height: "100%", overflowY: "auto", positio
    {!isPremium && <PremiumLock feature="Le mode analyse en direct est réservé
    <canvas ref={canvasRef} style={{ display: "none" }} />
    <div style={{ position: "relative", width: "100%", height: 260, background:
      <video ref={videoRef} autoPlay playsInline muted style={{ width: "100%",
      {liveActive && (
        <div style={{ position: "absolute", top: 12, left: 12, background: "rgb
          <div style={{ width: 6, height: 6, borderRadius: "50%", background: "
          <span style={{ fontSize: 11, color: "#fff", fontFamily: "'Bebas Neue'
        </div>
      )}
      <button onClick={() => { stopLive(); setMode("menu"); }} style={{ positio
    </div>
    <div style={{ padding: "14px 16px 0" }}>
      <div style={{ fontSize: 11, color: "#a855f7", fontFamily: "'Bebas Neue',
      {selectedCombo.refVideoUrl && (
        <div style={{ marginBottom: 10 }}>
          <div style={{ fontSize: 10, color: "#f59e0b", marginBottom: 6 }}>📹 R
          <video src={selectedCombo.refVideoUrl} controls style={{ width: "100%
        </div>
      )}
      <button onClick={captureAndAnalyzeLive} disabled={!liveActive || liveAnal
        {liveAnalyzing ? "⏳ ANALYSE..." : "📸 CAPTURER ET ANALYSER"}
```

```
      </button>
      {liveAnalysis && (
        <div style={{ ...S.card, background: liveAnalysis.posture === "bon" ? "
          <div style={{ fontSize: 11, color: liveAnalysis.posture === "bon" ? "
            {liveAnalysis.posture === "bon" ? "✅ POSTURE OK" : "⚠ À CORRIGER"}
          </div>
          <div style={{ fontSize: 13, color: "#fff", marginBottom: 6 }}>🔧 {liv
          <div style={{ fontSize: 12, color: "#aaa", fontStyle: "italic" }}>💡
        </div>
      )}
      <div style={{ ...S.card, marginTop: 12 }}>
        <div style={{ ...S.label, color: "#666" }}>POINTS CLÉS</div>
        {selectedCombo.cues.map((c, i) => <div key={i} style={{ fontSize: 11, c
      </div>
    </div>
  </div>
);

if (mode === "video") return (
  <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto" }}
    <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom:
      <button onClick={() => { setMode("menu"); setAnalysis(null); setVideoFile
      <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color: "
    </div>

    {/* Référence coach */}
    {selectedCombo.refVideoUrl && (
      <div style={{ marginBottom: 14 }}>
        <div style={{ fontSize: 10, color: "#f59e0b", fontFamily: "'Bebas Neue'
        <video src={selectedCombo.refVideoUrl} controls style={{ width: "100%",
      </div>
    )}

    <div style={{ background: "rgba(168,85,247,0.1)", border: "1px solid rgba(1
      <span style={{ fontSize: 12, color: "#a855f7", fontFamily: "'Bebas Neue',
    </div>

    <div onClick={() => fileInputRef.current?.click()} style={{border: `2px das
      <input ref={fileInputRef} type="file" accept="video/*,image/*" onChange={
      {videoPreview ? (
        videoFile?.type?.startsWith("video") ?
          <video src={videoPreview} controls style={{ width: "100%", borderRadi
          <img src={videoPreview} alt="preview" style={{ width: "100%", borderR
      ) : (
        <>
          <div style={{ fontSize: 36, marginBottom: 8 }}>📁</div>
          <div style={{ fontSize: 13, color: "#aaa", fontFamily: "'Bebas Neue',
```

```
      <div style={{ fontSize: 11, color: "#555", marginTop: 4 }}>Vidéo ou p
    </>
  )}
</div>

<textarea placeholder="Notes pour le coach (optionnel)..." value={coachNote
  style={{ ...S.input, resize: "none", height: 60, marginBottom: 12 }} />

<button onClick={analyzeVideo} disabled={!videoFile || analyzing} style={{
  {analyzing ? "⏳ ANALYSE EN COURS..." : "🤖 ANALYSER AVEC L'IA"}
</button>

{analysis && !analysis.error && (
  <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
    <div style={{ ...S.card, background: "rgba(168,85,247,0.1)", border: "1
      <div style={{ ...S.label, color: "#a855f7" }}>SCORE TECHNIQUE</div>
      <div style={{ fontSize: 52, fontFamily: "'Bebas Neue', cursive", colo
        {analysis.score}<span style={{ fontSize: 20 }}>/100</span>
      </div>
      <div style={{ fontSize: 12, color: "#ccc", marginTop: 4, fontStyle: "
    </div>
    {analysis.points_positifs?.length > 0 && (
      <div style={{ ...S.card, background: "rgba(34,197,94,0.07)", border:
        <div style={{ ...S.label, color: "#22c55e" }}>✅ POINTS POSITIFS</d
        {analysis.points_positifs.map((p, i) => <div key={i} style={{ fontS
      </div>
    )}
    {analysis.corrections?.length > 0 && (
      <div style={{ ...S.card, background: "rgba(239,68,68,0.07)", border:
        <div style={{ ...S.label, color: "#ef4444" }}>🔧 CORRECTIONS</div>
        {analysis.corrections.map((c, i) => (
          <div key={i} style={{ marginBottom: 12, paddingBottom: 10, border
            <div style={{ fontSize: 13, color: "#ef4444", fontFamily: "'Beb
            <div style={{ fontSize: 11, color: "#888", marginTop: 2 }}>⚠ {c
            <div style={{ fontSize: 12, color: "#ccc", marginTop: 2 }}>→ {c
          </div>
        ))}
      </div>
    )}
    {analysis.conseil_global && (
      <div style={{ ...S.card, background: "rgba(168,85,247,0.07)", border:
        <div style={{ ...S.label, color: "#a855f7" }}>💡 CONSEIL DU COACH</
        <div style={{ fontSize: 12, color: "#ccc", fontStyle: "italic", lin
      </div>
    )}
    <div style={{ fontSize: 10, color: "#555", textAlign: "center" }}>✓ Ana
  </div>
```

```
    )}
    {analysis?.error && <div style={{ ...S.card, background: "rgba(239,68,68,0.
  </div>
);

return (
  <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto" }}
    <style>{`@keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}`}</style>

    {/* Header tabs */}
    <div style={{ display: "flex", gap: 6, marginBottom: 20 }}>
      {[
        { id: "analyse", label: "🤖 Coach IA" },
        { id: "combos", label: "🥊 Combos" },
        { id: "historique", label: "📊 Historique" },
      ].map(t => (
        <button key={t.id} onClick={() => setTab(t.id)} style={{flex: 1, backgr
      ))}
    </div>

    <div style={{ fontSize: 26, fontFamily: "'Bebas Neue', cursive", color: "#f
      Analyse ta <span style={{ color: "#a855f7" }}>technique</span>
    </div>
    <div style={{ fontSize: 12, color: "#888", marginBottom: !isPremium ? 12 :
      L'IA corrige tes mouvements comme un vrai coach
    </div>

    {/* Compteur analyses gratuit */}
    {!isPremium && (
      <div onClick={goPaywall} style={{background: dailyAnalyses >= 1 ? "rgba(2
        <div>
          <div style={{ fontSize: 12, color: dailyAnalyses >= 1 ? "#ef4444" : "
            {dailyAnalyses >= 1 ? "🔒 LIMITE ATTEINTE" : "🎁 VERSION GRATUITE"}
          </div>
          <div style={{ fontSize: 10, color: "#666", marginTop: 2 }}>
            {dailyAnalyses >= 1 ? "Passe Premium pour analyser sans limite" : `
          </div>
        </div>
        <span style={{ fontSize: 11, color: "#f59e0b", fontFamily: "'Bebas Neue
      </div>
    )}

    {/* Sélection combo */}
    <div style={{ ...S.label, color: "#a855f7" }}>COMBO SÉLECTIONNÉ</div>
    <div style={{ display: "flex", flexDirection: "column", gap: 8, marginBotto
      {customCombos.map(c => (
        <button key={c.id} onClick={() => setSelectedComboId(c.id)} style={{bac
```

```
            <div>
              <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", co
              <div style={{ fontSize: 10, color: "#555", marginTop: 2 }}>{c.steps
            </div>
            {c.refVideoUrl && <span style={{ fontSize: 14 }}>🎥</span>}
          </button>
        ))}
        <button onClick={() => setTab("combos")} style={{ ...S.btn("rgba(168,85,2
          + CRÉER UN NOUVEAU COMBO
        </button>
      </div>

      {/* Modes */}
      <div style={{ ...S.label, color: "#a855f7" }}>CHOISIR UN MODE</div>
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10 }}>
        <button onClick={() => setMode("video")} style={{background: "rgba(168,85
          <span style={{ fontSize: 30 }}>🎬</span>
          <span style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", color
          <span style={{ fontSize: 10, color: "#666", lineHeight: 1.4, textAlign:
        </button>
        <button onClick={() => { setMode("live"); startLive(); }} style={{backgro
          <span style={{ fontSize: 30 }}>📹</span>
          <span style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", color
          <span style={{ fontSize: 10, color: "#666", lineHeight: 1.4, textAlign:
        </button>
      </div>

      {/* Stats rapides historique */}
      {history.length > 0 && (
        <div style={{ marginTop: 20 }}>
          <div style={{ ...S.label, color: "#22c55e" }}>DERNIÈRE ANALYSE</div>
          <div onClick={() => { setTab("historique"); setShowDetailHistory(histor
            <div style={{width: 48, height: 48, borderRadius: 12, flexShrink: 0,
            <div>
              <div style={{ fontSize: 12, fontFamily: "'Bebas Neue', cursive", co
              <div style={{ fontSize: 10, color: "#666" }}>{history[0].date} · Vo
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
// ──────────────────────────────────────────────────────────────────────

function NavBar({ active, setScreen }) {
  const icons = [
```

```
    { id:"home",        icon:"⊞",  label:"Accueil" },
    { id:"programs",    icon:"📋", label:"Programmes" },
    { id:"techniques", icon:"🥊", label:"Techniques" },
    { id:"timer",       icon:"⏱",  label:"Timer" },
    { id:"coach",       icon:"🤖", label:"Coach IA" },
    { id:"profile",     icon:"👤", label:"Profil" },
  ];
  return (
    <div style={{position:"absolute", bottom:0, left:0, right:0, background:"rgba
      {icons.map(i => {
        const isActive = active === i.id;
        return (
          <button key={i.id} onClick={() => setScreen(i.id)} style={{background:"
            {isActive && (
              <div style={{position:"absolute", top:-10, width:32, height:2, back
            )}
            <div style={{width:36, height:36, borderRadius:10, display:"flex", al
              <span style={{ fontSize:18 }}>{i.icon}</span>
            </div>
            <span style={{fontSize:9, fontFamily:D.heading, letterSpacing:1, colo
          </button>
        );
      })}
    </div>
  );
}

// ── DAILY QUOTES ──────────────────────────────────────────────
const QUOTES = [
  { text: "La douleur est temporaire. La gloire est éternelle.", author: "Muay Th
  { text: "Chaque round te rapproche de ta meilleure version.", author: "Autam" }
  { text: "L'entraînement d'aujourd'hui est la victoire de demain.", author: "Mua
  { text: "Ce n'est pas la taille du guerrier qui compte, mais sa volonté.", auth
  { text: "Discipline. Détermination. Dépassement.", author: "Autam Muay Thai" },
  { text: "Le champion s'entraîne quand personne ne regarde.", author: "Muay Thai
  { text: "Forge ton corps. Affûte ton esprit.", author: "Autam Muay Thai" },
];

const getDailyQuote = () => QUOTES[new Date().getDay() % QUOTES.length];
const getDayStreak = () => parseInt(localStorage.getItem("amt_streak") || "0");
const getTotalSessions = () => parseInt(localStorage.getItem("amt_sessions") || "
const getWeeklySessions = () => {
  try { return JSON.parse(localStorage.getItem("amt_weekly") || "[0,0,0,0,0,0,0]"
  catch { return [0, 0, 0, 0, 0, 0, 0]; }
};

const AI_WORKOUTS = [
```

```
  { title: "Cardio Fondamental", rounds: 5, duration: 3, rest: 1, level: "Débutan
  { title: "Puissance & Vitesse", rounds: 6, duration: 3, rest: 60, level: "Inter
  { title: "Guerrier Elite", rounds: 8, duration: 3, rest: 45, level: "Avancé", t
  { title: "Récupération Active", rounds: 4, duration: 2, rest: 90, level: "Tous
];

function HomeScreen({ setScreen, isPremium, user, trialDaysLeft }) {
  const [visible, setVisible] = useState(false);
  const [quoteVisible, setQuoteVisible] = useState(false);
  const quote = getDailyQuote();
  const streak = getDayStreak();
  const totalSessions = getTotalSessions();
  const weekly = getWeeklySessions();
  const today = new Date().getDay();
  const userName = user?.name?.split(" ")[0] || user?.email?.split("@")[0] || "Wa
  const aiWorkout = AI_WORKOUTS[new Date().getDay() % AI_WORKOUTS.length];

  useEffect(() => {
    setTimeout(() => setVisible(true), 80);
    setTimeout(() => setQuoteVisible(true), 400);
  }, []);

  const dayLabels = ["D", "L", "M", "M", "J", "V", "S"];

  return (
    <div style={{ padding: "0 0 90px", overflowY: "auto", height: "100%", backgro
      <style>{`
        @keyframes fadeUp { from { opacity:0; transform:translateY(16px); } to {
        @keyframes pulse2 { 0%,100%{transform:scale(1)} 50%{transform:scale(1.04)
        @keyframes shimmer { 0%{opacity:0.6} 50%{opacity:1} 100%{opacity:0.6} }
      `}</style>

      {/* ── HERO ── */}
      <div style={{position: "relative", height: 220, background: "#0a0a0a", over
        {/* Background line art */}
        <svg style={{ position: "absolute", inset: 0, width: "100%", height: "100
          <circle cx="320" cy="40" r="120" fill="none" stroke="#ef4444" strokeWid
          <circle cx="320" cy="40" r="80" fill="none" stroke="#fff" strokeWidth="
        </svg>

        {/* Greeting */}
        <div style={{opacity: visible ? 1 : 0, transform: visible ? "none" : "tra
          <div style={{ fontSize: 12, color: "#555", letterSpacing: 2, fontFamily
            {new Date().toLocaleDateString("fr-FR", { weekday: "long", day: "nume
          </div>
          <div style={{ fontSize: 28, fontFamily: "'Bebas Neue', cursive", color:
            BON RETOUR,<br />
```

```
      <span style={{ color: "#ef4444" }}>{userName.toUpperCase()}</span>
    </div>
  </div>

  {/* Streak badge */}
  {streak > 0 && (
    <div style={{position: "absolute", top: 16, right: 16, background: "rgb
      <span style={{ fontSize: 14 }}>🔥</span>
      <span style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", col
        {streak} JOURS
      </span>
    </div>
  )}

  {isPremium && trialDaysLeft > 0 && (
    <div style={{position: "absolute", top: streak > 0 ? 56 : 16, right: 16
      <span style={{ fontSize: 10, fontFamily: "'Bebas Neue', cursive", col
        🎁 {trialDaysLeft}J ESSAI
      </span>
    </div>
  )}
</div>

{/* ── CITATION DU JOUR ── */}
<div style={{margin: "16px 16px 0", opacity: quoteVisible ? 1 : 0, transfor
  <div style={{background: "rgba(255,255,255,0.02)", borderLeft: "3px solid
    <div style={{ fontSize: 13, color: "#ddd", fontStyle: "italic", lineHei
      "{quote.text}"
    </div>
    <div style={{ fontSize: 10, color: "#555", letterSpacing: 2, fontFamily
      — {quote.author}
    </div>
  </div>
</div>

{/* ── ACTIVITÉ SEMAINE ── */}
<div style={{ padding: "20px 16px 0" }}>
  <div style={{ display: "flex", justifyContent: "space-between", alignItem
    <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFam
      CETTE SEMAINE
    </div>
    <div style={{ fontSize: 11, color: "#666" }}>{totalSessions} séances au
  </div>
  <div style={{background: "rgba(255,255,255,0.02)", borderRadius: 16, bord
    <div style={{ display: "flex", alignItems: "flex-end", gap: 6, height:
      {dayLabels.map((d, i) => {
        const isToday = i === today;
```

```
        const val = weekly[i] || 0;
        const h = Math.max(4, (val / Math.max(...weekly, 1)) * 42);
        return (
          <div key={i} style={{ flex: 1, display: "flex", flexDirection: "c
            <div style={{width: "100%", borderRadius: 4, height: `${h}px`,
            <span style={{ fontSize: 9, color: isToday ? "#ef4444" : "#555"
          </div>
        );
      })}
    </div>
    {/* Stats rapides */}
    <div style={{ display: "flex", gap: 12, borderTop: "1px solid rgba(255,
      {[
        { icon: "🔥", label: `${streak} jours streak` },
        { icon: "⏱️", label: `${totalSessions * 45}min total` },
        { icon: "💪", label: `${totalSessions} séances` },
      ].map(s => (
        <div key={s.label} style={{ display: "flex", alignItems: "center",
          <span style={{ fontSize: 11 }}>{s.icon}</span>
          <span style={{ fontSize: 10, color: "#666" }}>{s.label}</span>
        </div>
      ))}
    </div>
  </div>
</div>

{/* ── ENTRAÎNEMENT RECOMMANDÉ PAR L'IA ── */}
<div style={{ padding: "20px 16px 0" }}>
  <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFamil
    RECOMMANDÉ PAR L'IA AUJOURD'HUI
  </div>
  <div style={{background: `linear-gradient(135deg, ${aiWorkout.color}18, $
    <div style={{ display: "flex", justifyContent: "space-between", alignIt
      <div>
        <div style={{ fontSize: 20, fontFamily: "'Bebas Neue', cursive", co
          {aiWorkout.title}
        </div>
        <div style={{ fontSize: 11, color: "#888", marginTop: 2 }}>
          {aiWorkout.rounds} rounds · {aiWorkout.duration} min · {aiWorkout
        </div>
      </div>
      <span style={{fontSize: 10, color: aiWorkout.color, border: `1px soli
    </div>
    <div style={{ display: "flex", gap: 6, flexWrap: "wrap", marginBottom:
      {aiWorkout.tags.map(t => (
        <span key={t} style={{fontSize: 10, color: "#aaa", background: "rgb
      ))}
```

```
          </div>
          <button onClick={() => setScreen("timer")} style={{width: "100%", borde
        </div>
      </div>

      {/* ── ACCÈS RAPIDE ── */}
      <div style={{ padding: "20px 16px 0" }}>
        <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFamil
          ACCÈS RAPIDE
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 }}>
          {[
            { icon: "💬", label: "Chat Coach", screen: "chat", accent: "#ef4444",
            { icon: "⚡", label: "Défi du Jour", screen: "defi", accent: "#f59e0b
            { icon: "🤖", label: "Coach IA", screen: "coach", accent: "#a855f7",
            { icon: "⏱️", label: "Timer", screen: "timer", accent: "#22c55e", sub
          ].map((a, i) => (
            <button key={a.label} onClick={() => setScreen(a.screen)} style={{bac
              <span style={{ fontSize: 22 }}>{a.icon}</span>
              <div>
                <div style={{ fontSize: 13, color: "#fff", fontFamily: "'Bebas Ne
                <div style={{ fontSize: 10, color: "#555", marginTop: 2 }}>{a.sub
              </div>
              <div style={{ width: 20, height: 2, background: a.accent, borderRad
            </button>
          ))}
        </div>
      </div>

      {/* ── MOTIVATION FINALE ── */}
      <div style={{ padding: "20px 16px 0" }}>
        <div style={{background: "#0a0a0a", border: "1px solid rgba(255,255,255,0
          <div style={{width: 44, height: 44, borderRadius: 12, flexShrink: 0, ba
          <div>
            <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", colo
              CONTINUE COMME ÇA
            </div>
            <div style={{ fontSize: 11, color: "#555", marginTop: 2, lineHeight:
              Chaque séance compte. Tu construis quelque chose de grand.
            </div>
          </div>
        </div>
      </div>

    </div>
  );
}
```

```
// ── Audio helpers ──────────────────────────────────────────────
function createBeep(ctx, freq, duration, gain = 0.4) {
  const osc = ctx.createOscillator();
  const vol = ctx.createGain();
  osc.connect(vol);
  vol.connect(ctx.destination);
  osc.type = "square";
  osc.frequency.setValueAtTime(freq, ctx.currentTime);
  vol.gain.setValueAtTime(gain, ctx.currentTime);
  vol.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + duration);
  osc.start(ctx.currentTime);
  osc.stop(ctx.currentTime + duration);
}

function playWarningBeep(ctx) {
  [0, 0.22, 0.44].forEach(delay => {
    const osc = ctx.createOscillator();
    const vol = ctx.createGain();
    osc.connect(vol); vol.connect(ctx.destination);
    osc.type = "square"; osc.frequency.value = 880;
    vol.gain.setValueAtTime(0.35, ctx.currentTime + delay);
    vol.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + delay + 0.18);
    osc.start(ctx.currentTime + delay);
    osc.stop(ctx.currentTime + delay + 0.18);
  });
}

function playEndGong(ctx) {
  const osc1 = ctx.createOscillator();
  const osc2 = ctx.createOscillator();
  const vol = ctx.createGain();
  osc1.connect(vol); osc2.connect(vol); vol.connect(ctx.destination);
  osc1.type = "sine"; osc1.frequency.setValueAtTime(120, ctx.currentTime);
  osc1.frequency.exponentialRampToValueAtTime(60, ctx.currentTime + 1.2);
  osc2.type = "sine"; osc2.frequency.setValueAtTime(180, ctx.currentTime);
  osc2.frequency.exponentialRampToValueAtTime(90, ctx.currentTime + 1.2);
  vol.gain.setValueAtTime(0.8, ctx.currentTime);
  vol.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 1.8);
  osc1.start(ctx.currentTime); osc1.stop(ctx.currentTime + 1.8);
  osc2.start(ctx.currentTime); osc2.stop(ctx.currentTime + 1.8);
}

function playRestEnd(ctx) {
  [523, 659, 784].forEach((freq, i) => {
    const osc = ctx.createOscillator();
    const vol = ctx.createGain();
```

```
    osc.connect(vol); vol.connect(ctx.destination);
    osc.type = "sine"; osc.frequency.value = freq;
    vol.gain.setValueAtTime(0.5, ctx.currentTime + i * 0.15);
    vol.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + i * 0.15 + 0.4
    osc.start(ctx.currentTime + i * 0.15);
    osc.stop(ctx.currentTime + i * 0.15 + 0.4);
  });
}
// ────────────────────────────────────────────────────────────────────────────

function TimerScreen({ onSessionComplete }) {
  const [rounds, setRounds] = useState(5);
  const [roundTime, setRoundTime] = useState(180);
  const [restTime, setRestTime] = useState(60);
  const [running, setRunning] = useState(false);
  const [phase, setPhase] = useState("round");
  const [currentRound, setCurrentRound] = useState(1);
  const [timeLeft, setTimeLeft] = useState(180);
  const [flash, setFlash] = useState(false);
  const [sessionModal, setSessionModal] = useState(false);
  const [sessionMinutes, setSessionMinutes] = useState(0);
  const intervalRef = useRef(null);
  const audioCtxRef = useRef(null);
  const warningPlayedRef = useRef(false);

  const getAudioCtx = () => {
    if (!audioCtxRef.current) {
      audioCtxRef.current = new (window.AudioContext || window.webkitAudioContext
    }
    if (audioCtxRef.current.state === "suspended") {
      audioCtxRef.current.resume();
    }
    return audioCtxRef.current;
  };

  useEffect(() => {
    setTimeLeft(roundTime);
    warningPlayedRef.current = false;
  }, [roundTime]);

  useEffect(() => {
    if (running) {
      intervalRef.current = setInterval(() => {
        setTimeLeft(t => {
          const next = t - 1;

          if (next === 10 && !warningPlayedRef.current) {
```

```
          warningPlayedRef.current = true;
          playWarningBeep(getAudioCtx());
        }

        if (next <= 0) {
          warningPlayedRef.current = false;
          setFlash(true);
          setTimeout(() => setFlash(false), 600);

          if (phase === "round") {
            if (currentRound >= rounds) {
              playEndGong(getAudioCtx());
              setRunning(false);
              setCurrentRound(1);
              setPhase("round");
              setTimeLeft(roundTime);
              const mins = Math.round((rounds * roundTime) / 60);
              setSessionMinutes(mins);
              setTimeout(() => setSessionModal(true), 800);
              return 0;
            }
            playEndGong(getAudioCtx());
            setPhase("repos");
            return restTime;
          } else {
            playRestEnd(getAudioCtx());
            setPhase("round");
            setCurrentRound(r => r + 1);
            return roundTime;
          }
        }
        return next;
      });
    }, 1000);
  }
  return () => clearInterval(intervalRef.current);
}, [running, phase, currentRound, rounds, roundTime, restTime]);

const reset = () => {
  setRunning(false);
  clearInterval(intervalRef.current);
  setCurrentRound(1);
  setPhase("round");
  setTimeLeft(roundTime);
  warningPlayedRef.current = false;
};
```

```
const handleStartStop = () => {
  getAudioCtx(); // unlock audio on tap
  setRunning(r => !r);
};

const fmt = s => `${String(Math.floor(s / 60)).padStart(2, "0")}:${String(s % 6
const progress = phase === "round" ? timeLeft / roundTime : timeLeft / restTime
const circumference = 2 * Math.PI * 90;
const isWarning = timeLeft <= 10 && timeLeft > 0 && running;

return (
  <div style={{ padding: "20px 16px 80px", height: "100%", overflowY: "auto", p
    {sessionModal && (
      <SessionCompleteModal
        minutes={sessionMinutes}
        onClose={() => setSessionModal(false)}
        onBadge={onSessionComplete || (() => {})}
      />
    )}
    {/* Flash overlay on end */}
    {flash && (
      <div style={{position: "absolute", inset: 0, zIndex: 50, pointerEvents: "
    )}
    <style>{`@keyframes flashFade { from { opacity:1 } to { opacity:0 } } @keyf

    <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFamily:
      TIMER DE ROUND
    </div>

    {/* Phase badge */}
    <div style={{ textAlign: "center", marginBottom: 10 }}>
      <span style={{fontSize: 13, letterSpacing: 3, fontFamily: "'Bebas Neue',
        {phase === "round" ? `ROUND ${currentRound} / ${rounds}` : "⏸️ REPOS"}
      </span>
    </div>

    {/* 10s warning label */}
    {isWarning && (
      <div style={{textAlign: "center", marginBottom: 6, fontSize: 12, color: "
        ⚠ 10 SECONDES !
      </div>
    )}

    {/* Circle timer */}
    <div style={{ display: "flex", justifyContent: "center", marginBottom: 24 }
      <div style={{ position: "relative", width: 220, height: 220 }}>
        <svg width="220" height="220" style={{ transform: "rotate(-90deg)" }}>
```

```
          <circle cx="110" cy="110" r="90" fill="none" stroke="rgba(255,255,255
          <circle cx="110" cy="110" r="90" fill="none"
            stroke={isWarning ? "#f59e0b" : phase === "round" ? "#ef4444" : "#2
            strokeWidth="8" strokeLinecap="round"
            strokeDasharray={circumference}
            strokeDashoffset={circumference * (1 - progress)}
            style={{ transition: "stroke-dashoffset 0.5s ease, stroke 0.3s" }}
          />
        </svg>
        <div style={{position: "absolute", inset: 0, display: "flex", flexDirec
          <div style={{fontSize: 52, fontFamily: "'Bebas Neue', cursive", color
          <div style={{ fontSize: 11, color: "#666", marginTop: 4 }}>
            {phase === "round" ? "en cours" : "repos"}
          </div>
        </div>
      </div>
    </div>

    {/* Controls */}
    <div style={{ display: "flex", justifyContent: "center", gap: 16, marginBot
      <button onClick={reset} style={{background: "rgba(255,255,255,0.06)", bor
      <button onClick={handleStartStop} style={{background: running ? "rgba(220
      <button onClick={() => { reset(); }} style={{background: "rgba(255,255,25
    </div>

    {/* Settings */}
    <div style={{ background: "rgba(255,255,255,0.03)", border: "1px solid rgba
      <div style={{ fontSize: 11, color: "#666", letterSpacing: 2, fontFamily:
      {[
        { label: "Rounds", value: rounds, set: setRounds, min: 1, max: 12, step
        { label: "Durée du round", value: roundTime, set: v => { setRoundTime(v
        { label: "Temps de repos", value: restTime, set: setRestTime, min: 15,
      ].map(s => (
        <div key={s.label} style={{ marginBottom: 14 }}>
          <div style={{ display: "flex", justifyContent: "space-between", margi
            <span style={{ fontSize: 12, color: "#aaa" }}>{s.label}</span>
            <span style={{ fontSize: 12, color: "#ef4444", fontFamily: "'Bebas
          </div>
          <input type="range" min={s.min} max={s.max} step={s.step} value={s.va
            onChange={e => s.set(Number(e.target.value))}
            disabled={running}
            style={{ width: "100%", accentColor: "#ef4444" }}
          />
        </div>
      ))}
    </div>
  </div>
```

```
  );
}

// ── RICH CONTENT DATA ──────────────────────────────────────────
const RICH_TECHNIQUES = [
  { id:1, name:"Jab", thai:"หมัดตรงซ้าย", category:"Poing", emoji:"👊", level:"Débu
    desc:"Le jab est le coup de base du Muay Thai. Rapide et direct, il sert à me
    steps:["Position en garde — pied gauche devant, mains hautes","Projeter le po
    keys:["Garde haute de l'autre main","Pivot léger sur le talon avant","Frappe
    errors:["Baisser la garde de protection","Telegraphier le coup en abaissant l

  { id:2, name:"Cross (Droit)", thai:"หมัดตรงขวา", category:"Poing", emoji:"👊", le
    desc:"Le cross est le coup de poing puissant du poing arrière. C'est l'arme p
    steps:["Partir de la garde — pied droit derrière","Pivoter sur la pointe du p
    keys:["Rotation totale des hanches = puissance maximale","Pivot sur la pointe
    errors:["Pas assez de rotation des hanches","Lever le talon arrière trop tôt"

  { id:3, name:"Crochet", thai:"หมัดเหวี่ยง", category:"Poing", emoji:"👊", level:"In
    desc:"Le crochet frappe sur le côté de la mâchoire ou du corps. Très efficace
    steps:["Coude à 90° — bras parallèle au sol","Rotation explosive du tronc et
    keys:["Coude horizontal — pas en diagonale","Poids sur le pied avant pour plu
    errors:["Coude trop haut ou trop bas","Mouvement trop large et prévisible","O

  { id:4, name:"Teep (Coup de pied frontal)", thai:"ถีบ", category:"Coup de pied",
    desc:"Le Teep est le jab des pieds en Muay Thai. Il sert à contrôler la dista
    steps:["Lever le genou haut vers la cible","Étendre la jambe rapidement et pu
    keys:["Lever le genou avant d'étendre — génère de la puissance","Frapper avec
    errors:["Frapper avec le bout du pied","Pas assez de montée de genou","Perdre

  { id:5, name:"Roundhouse Kick", thai:"เตะตัด", category:"Coup de pied", emoji:"🦵
    desc:"L'arme signature du Muay Thai. Circulaire et dévastateur, ce kick cible
    steps:["Pivot sur le pied avant — talon vers la cible","Rotation explosive de
    keys:["Tibia = arme, pas le pied","Rotation complète des hanches obligatoire"
    errors:["Frapper avec le pied au lieu du tibia","Pas assez de rotation des ha

  { id:6, name:"Coude Horizontal", thai:"ศอกตัด", category:"Coude", emoji:"💥", le
    desc:"Le coude horizontal est une frappe dévastatrice à courte distance. Très
    steps:["Position rapprochée de l'adversaire","Lever le coude à hauteur de têt
    keys:["Distance courte = coude efficace","Viser la tempe, l'arcade ou la mâch
    errors:["Trop loin de l'adversaire","Coude trop bas — frappe avec l'avant-bra

  { id:7, name:"Coude Montant", thai:"ศอกงัด", category:"Coude", emoji:"💥", level
    desc:"Le coude montant frappe de bas en haut, idéal pour l'uppercut version M
    steps:["Partir du bas — coude pointé vers le bas","Mouvement ascendant explos
    keys:["Genoux légèrement fléchis pour charger","Explosion des jambes vers le
    errors:["Ne pas utiliser les jambes","Distance trop grande","Angle incorrect
```

```
  { id:8, name:"Genou Droit", thai:"เข่าตรง", category:"Genou", emoji:"🔥", level:"
    desc:"Le genou droit est une arme redoutable en clinch. Monté puissamment, il
    steps:["Saisir la nuque ou les épaules adverse (clinch)","Tirer la tête de l'
    keys:["Clinch solide pour contrôler l'adversaire","Tirer la tête vers le geno
    errors:["Clinch trop faible — adversaire s'échappe","Se pencher en arrière —

  { id:9, name:"Genou Sauté", thai:"เข่าลอย", category:"Genou", emoji:"🔥", level:'
    desc:"Le genou sauté est spectaculaire et dévastateur. Il permet de passer pa
    steps:["Élan et saut explosif sur un pied","Monter le genou en hauteur maxima
    keys:["Vitesse d'approche = puissance du genou","Viser haut pour dépasser la
    errors:["Pas assez d'élan","Atterrir sans équilibre","Trop prévisible — combi

  { id:10, name:"Parade Haute", thai:"การป้องกัน", category:"Défense", emoji:"🛡️", 
    desc:"La parade haute protège la tête des coups de poing et de certains kicks
    steps:["Coudes serrés contre les tempes","Avant-bras parallèles devant le vis
    keys:["Coudes serrés = protection maximale","Ne pas fermer les yeux — voir ve
    errors:["Coudes trop écartés","Fermer les yeux","Rester passif — toujours con

  { id:11, name:"Esquive Extérieure", thai:"การหลบ", category:"Défense", emoji:"🛡
    desc:"L'esquive extérieure permet d'éviter un coup tout en se plaçant pour co
    steps:["Détecter le coup entrant","Pivoter sur le pied avant vers l'extérieur
    keys:["Mouvement des pieds = clé de l'esquive","Garder les yeux sur l'adversa
    errors:["Mouvement trop grand — perd l'équilibre","Fermer les yeux","Oublier

  { id:12, name:"Clinch (Corps à Corps)", thai:"การจับคอ", category:"Clinch", emoj
    desc:"Le clinch est une spécialité du Muay Thai. C'est la maîtrise du corps à
    steps:["Saisir la nuque avec les deux mains","Croiser les poignets derrière l
    keys:["Mains derrière la nuque — pas sur les épaules","Contrôle de la tête =
    errors:["Saisir les épaules — moins de contrôle","Bras pas croisés — adversai
];

const RICH_PROGRAMS = [
  {
    id:"p1", name:"Fondamentaux du Muay Thai", weeks:4, level:"Débutant", session
    desc:"Programme idéal pour les débutants. Tu vas apprendre les bases du Muay
    objectif:"Maîtriser la garde, le jab, le cross et le teep. Développer l'endur
    weekly: [
      { week:1, title:"Semaine 1 — La Garde et les Bases", sessions:[
        { day:"Lundi", title:"Découverte", duration:45, exercises:["Échauffement
        { day:"Mercredi", title:"Technique", duration:45, exercises:["Échauffemen
        { day:"Vendredi", title:"Cardio", duration:45, exercises:["Corde à sauter
      ]},
      { week:2, title:"Semaine 2 — Introduction aux Kicks", sessions:[
        { day:"Lundi", title:"Low Kick", duration:50, exercises:["Échauffement","
        { day:"Mercredi", title:"Mid Kick", duration:50, exercises:["Échauffement
        { day:"Vendredi", title:"Consolidation", duration:55, exercises:["5 round
      ]},
```

```
    ],
    nutrition:"Protéines 1.8g/kg, glucides complexes avant l'entraînement, hydrat
    recuperation:"Étirements 15min après chaque séance, glace sur les zones doulo
  },
  {
    id:"p2", name:"Combat Conditioning", weeks:6, level:"Intermédiaire", sessions
    desc:"Programme pour développer ta puissance, ta vitesse et ton endurance de
    objectif:"Maîtriser les combinaisons avancées, développer la puissance des fr
    weekly:[
      { week:1, title:"Semaine 1 — Puissance", sessions:[
        { day:"Lundi", title:"Poings et vitesse", duration:60, exercises:["Échauf
        { day:"Mardi", title:"Kicks et tibia", duration:60, exercises:["Condition
        { day:"Jeudi", title:"Coudes et genoux", duration:60, exercises:["Coude h
        { day:"Samedi", title:"Sparring technique", duration:60, exercises:["Écha
      ]},
    ],
    nutrition:"Protéines 2g/kg, repas pré-entraînement 2h avant, créatine optionn
    recuperation:"Massage 2x/semaine, bain froid après séances intenses, sommeil
  },
  {
    id:"p3", name:"Guerrier Élite", weeks:8, level:"Avancé", sessions:5, color:"#
    desc:"Programme de préparation combat niveau élite. Intensité maximale, techn
    objectif:"Atteindre un niveau de compétition. Maîtriser toutes les armes du M
    weekly:[
      { week:1, title:"Semaine 1 — Fondation Elite", sessions:[
        { day:"Lundi", title:"Puissance explosive", duration:90, exercises:["Cond
        { day:"Mardi", title:"Technique et vitesse", duration:75, exercises:["Sha
        { day:"Mercredi", title:"Clinch et corps à corps", duration:75, exercises
        { day:"Vendredi", title:"Sparring complet", duration:90, exercises:["Écha
        { day:"Samedi", title:"Cardio et récup", duration:60, exercises:["Jogging
      ]},
    ],
    nutrition:"Plan nutritionnel strict, timing des repas précis, supplémentation
    recuperation:"Récupération = partie de l'entraînement. Cryothérapie, massages
  },
];

const GLOSSAIRE = [
  { thai:"มวยไทย", roman:"Muay Thai", fr:"Boxe Thaïlandaise", desc:"L'art des 8 me
  { thai:"วาย", roman:"Wai Kru", fr:"Salut au maître", desc:"Rituel d'avant combat
  { thai:"ครู", roman:"Kru", fr:"Professeur", desc:"Titre donné au coach ou maître
  { thai:"ยก", roman:"Yok", fr:"Round", desc:"Période de combat (généralement 3 m
  { thai:"หมัด", roman:"Mud", fr:"Poing", desc:"Coup de poing en général" },
  { thai:"เตะ", roman:"Tae", fr:"Coup de pied", desc:"Kick en général" },
  { thai:"ศอก", roman:"Sok", fr:"Coude", desc:"Frappe de coude" },
  { thai:"เข่า", roman:"Khao", fr:"Genou", desc:"Frappe de genou" },
  { thai:"ถีบ", roman:"Teep", fr:"Coup de pied frontal", desc:"Coup de pied de fac
```

```
  { thai:"เตะตัด", roman:"Tae Tad", fr:"Roundhouse Kick", desc:"Coup de pied circul
  { thai:"จับคอ", roman:"Chap Kho", fr:"Clinch", desc:"Corps à corps, spécialité d
  { thai:"หมัดตรง", roman:"Mud Trong", fr:"Direct", desc:"Coup de poing direct (jal
  { thai:"หมัดเหวี่ยง", roman:"Mud Wiang", fr:"Crochet", desc:"Coup de poing circulai
  { thai:"ป้องกัน", roman:"Pong Kan", fr:"Défense / Parade", desc:"Protection contr
  { thai:"ค่าย", roman:"Khai", fr:"Camp d'entraînement", desc:"Gym ou école de Mua
  { thai:"นักมวย", roman:"Nak Muay", fr:"Boxeur", desc:"Pratiquant de Muay Thai" }
];

const NUTRITION_TIPS = [
  { icon:"🥩", title:"Protéines", color:"#ef4444", tips:[
    "1.8 à 2.2g de protéines par kg de poids corporel",
    "Répartis sur 4-5 repas dans la journée",
    "Sources : poulet, thon, œufs, légumineuses",
    "Whey protéine optionnelle après l'entraînement",
  ]},
  { icon:"🍚", title:"Glucides", color:"#f59e0b", tips:[
    "Carburant principal pour l'entraînement intense",
    "Repas riche en glucides 2-3h avant la séance",
    "Riz, patate douce, avoine, quinoa",
    "Réduire après 18h si pas d'entraînement le soir",
  ]},
  { icon:"🥑", title:"Lipides", color:"#22c55e", tips:[
    "Ne pas les éliminer — essentiels pour les hormones",
    "Sources saines : avocat, noix, huile d'olive",
    "20-30% des calories totales",
    "Oméga-3 anti-inflammatoires recommandés",
  ]},
  { icon:"💧", title:"Hydratation", color:"#a855f7", tips:[
    "2.5 à 3.5L d'eau par jour minimum",
    "Peser avant/après l'entraînement : 1kg perdu = 1L à boire",
    "Boissons isotoniques pour séances > 90min",
    "Urine claire = bonne hydratation",
  ]},
  { icon:"⏰", title:"Timing des Repas", color:"#06b6d4", tips:[
    "2-3h avant : repas complet (glucides + protéines)",
    "30-60min avant : snack léger si besoin",
    "Immédiatement après : protéines + glucides rapides",
    "2h après : repas complet de récupération",
  ]},
  { icon:"😴", title:"Récupération", color:"#8b5cf6", tips:[
    "Sommeil 8-9h — la croissance musculaire se fait la nuit",
    "Étirements 15-20min après chaque séance",
    "Bain froid ou glace sur zones douloureuses",
    "Jour de repos actif : marche, yoga, natation légère",
  ]},
];
```

```
function TechniquesScreen({ isPremium, goPaywall, initialTab }) {
  const [tab, setTab] = useState(initialTab || "techniques");
  const [selected, setSelected] = useState(null);
  const [catFilter, setCatFilter] = useState("Tous");
  const [progFilter, setProgFilter] = useState(null);
  const [selectedProg, setSelectedProg] = useState(null);
  const [glossSearch, setGlossSearch] = useState("");

  const categories = ["Tous", "Poing", "Coup de pied", "Coude", "Genou", "Défense
  const filtered = catFilter === "Tous" ? RICH_TECHNIQUES : RICH_TECHNIQUES.filte
  const levelColors = { "Débutant":"#22c55e","Intermédiaire":"#f59e0b","Avancé":"

  const tabs = [
    { id:"techniques", label:"🥊 Techniques" },
    { id:"programs",   label:"📋 Programmes" },
    { id:"nutrition",  label:"🥗 Nutrition" },
    { id:"glossaire",  label:"🇹🇭 Glossaire" },
  ];

  return (
    <div style={{ padding:"0 0 80px", height:"100%", overflowY:"auto", background
      {/* Header */}
      <div style={{ padding:"20px 16px 0", background:D.bg }}>
        <SLabel>CONTENU & APPRENTISSAGE</SLabel>
      </div>

      {/* Tabs */}
      <div style={{ display:"flex", gap:6, padding:"0 16px 16px", overflowX:"auto
        {tabs.map(t => (
          <button key={t.id} onClick={() => setTab(t.id)} style={{background: tab
        ))}
      </div>

      {/* ── TECHNIQUES ── */}
      {tab=="techniques" && (
        <div style={{ padding:"0 16px" }}>
          {/* Filtre catégorie */}
          <div style={{ display:"flex", gap:6, marginBottom:16, overflowX:"auto",
            {categories.map(c => (
              <button key={c} onClick={() => setCatFilter(c)} style={{background:
            ))}
          </div>

          {selected ? (
            <div style={{ animation:"scaleIn 0.3s ease" }}>
              <button onClick={() => setSelected(null)} style={{ background:D.bgC
```

```
  ← RETOUR
</button>
<div style={{ background:`${selected.color}10`, border:`1px solid $
  <div style={{ display:"flex", alignItems:"center", gap:14, margin
    <span style={{ fontSize:44 }}>{selected.emoji}</span>
    <div>
      <div style={{ fontSize:24, fontFamily:D.heading, color:D.text
      <div style={{ fontSize:11, color:D.textMuted, fontStyle:"ital
      <span style={{ fontSize:10, color:selected.color, border:`1px
    </div>
  </div>
  <p style={{ fontSize:13, color:D.textSecondary, lineHeight:1.7, m
</div>

{/* Étapes */}
<div style={{ background:D.bgCard, border:`1px solid ${D.border}`,
  <SLabel color={selected.color}>📋 ÉTAPES D'EXÉCUTION</SLabel>
  {selected.steps.map((s,i) => (
    <div key={i} style={{ display:"flex", gap:12, marginBottom:10,
      <div style={{ width:24, height:24, borderRadius:"50%", backgr
      <div style={{ fontSize:13, color:D.textSecondary, lineHeight:
    </div>
  ))}
</div>

{/* Points clés */}
<div style={{ background:D.greenSoft, border:"1px solid rgba(0,230,
  <SLabel color={D.green}>✅ POINTS CLÉS</SLabel>
  {selected.keys.map((k,i) => (
    <div key={i} style={{ display:"flex", gap:8, marginBottom:6 }}>
      <span style={{ color:D.green, fontSize:12, flexShrink:0 }}>→<
      <span style={{ fontSize:12, color:D.textSecondary, lineHeight
    </div>
  ))}
</div>

{/* Erreurs courantes */}
<div style={{ background:"rgba(255,26,26,0.06)", border:`1px solid
  <SLabel>⚠️ ERREURS À ÉVITER</SLabel>
  {selected.errors.map((e,i) => (
    <div key={i} style={{ display:"flex", gap:8, marginBottom:6 }}>
      <span style={{ color:D.red, fontSize:12, flexShrink:0 }}>✗</s
      <span style={{ fontSize:12, color:D.textSecondary, lineHeight
    </div>
  ))}
</div>
```

```
          <div style={{ background:D.bgCard, border:`1px solid ${D.border}`,
            <div style={{ fontSize:12, color:D.red }}>🎥 Vidéo de démonstrati
          </div>
        </div>
      ) : (
        <div style={{ display:"flex", flexDirection:"column", gap:8 }}>
          {filtered.map(t => (
            <div key={t.id} onClick={() => setSelected(t)} style={{ backgroun
              <span style={{ fontSize:28 }}>{t.emoji}</span>
              <div style={{ flex:1 }}>
                <div style={{ fontSize:14, fontFamily:D.heading, color:D.text
                <div style={{ fontSize:10, color:t.color, marginTop:2 }}>{t.c
                <div style={{ fontSize:10, color:D.textMuted, marginTop:1, fo
              </div>
              <div style={{ display:"flex", flexDirection:"column", alignItem
                <span style={{ fontSize:9, color:t.color, border:`1px solid $
                <span style={{ color:D.textMuted, fontSize:14 }}>›</span>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  )}

  {/* ── PROGRAMMES ── */}
  {tab=="programs" && (
    <div style={{ padding:"0 16px" }}>
      {selectedProg ? (
        <div style={{ animation:"scaleIn 0.3s ease" }}>
          <button onClick={() => setSelectedProg(null)} style={{ background:D
            ← RETOUR
          </button>
          <div style={{ background:`${selectedProg.color}10`, border:`1px sol
            <div style={{ fontSize:32, marginBottom:8 }}>{selectedProg.icon}<
            <div style={{ fontSize:24, fontFamily:D.heading, color:D.textPrim
            <div style={{ fontSize:12, color:D.textSecondary, marginBottom:10
            <div style={{ display:"flex", gap:8, flexWrap:"wrap" }}>
              {[
                { label:`${selectedProg.weeks} semaines`, icon:"📅" },
                { label:`${selectedProg.sessions}x/semaine`, icon:"💪" },
                { label:selectedProg.level, icon:"⚡" },
              ].map(s => (
                <span key={s.label} style={{ fontSize:11, color:selectedProg.
                  {s.icon} {s.label}
                </span>
              ))}
```

```
          </div>
        </div>

        <div style={{ background:D.bgCard, border:`1px solid ${D.border}`,
          <SLabel color={selectedProg.color}>🎯 OBJECTIF</SLabel>
          <p style={{ fontSize:13, color:D.textSecondary, lineHeight:1.6, m
        </div>

        {selectedProg.weekly?.map((w,wi) => (
          <div key={wi} style={{ background:D.bgCard, border:`1px solid ${D
            <SLabel color={selectedProg.color}>📅 {w.title}</SLabel>
            {w.sessions?.map((s,si) => (
              <div key={si} style={{ marginBottom:12, paddingBottom:10, bor
                <div style={{ display:"flex", justifyContent:"space-between
                  <div style={{ fontSize:13, fontFamily:D.heading, color:D.
                  <span style={{ fontSize:10, color:selectedProg.color }}>{
                </div>
                {s.exercises.map((e,ei) => (
                  <div key={ei} style={{ fontSize:11, color:D.textMuted, ma
                    <span style={{ color:selectedProg.color, flexShrink:0 }
                  </div>
                ))}
              </div>
            ))}
          </div>
        ))}

        <div style={{ background:"rgba(0,230,118,0.06)", border:"1px solid
          <SLabel color={D.green}>🥗 NUTRITION</SLabel>
          <p style={{ fontSize:12, color:D.textSecondary, lineHeight:1.7, m
        </div>

        <div style={{ background:"rgba(168,85,247,0.06)", border:"1px solid
          <SLabel color="#a855f7">😴 RÉCUPÉRATION</SLabel>
          <p style={{ fontSize:12, color:D.textSecondary, lineHeight:1.7, m
        </div>
      </div>
    ) : (
      <div style={{ display:"flex", flexDirection:"column", gap:12 }}>
        {RICH_PROGRAMS.map(p => (
          <div key={p.id} onClick={() => setSelectedProg(p)} style={{ backg
            <div style={{ position:"absolute", right:-10, top:-10, fontSize
            <div style={{ display:"flex", alignItems:"center", gap:14, marg
              <div style={{ width:52, height:52, borderRadius:14, backgroun
              <div style={{ flex:1 }}>
                <div style={{ fontSize:18, fontFamily:D.heading, color:D.te
                <div style={{ fontSize:11, color:D.textMuted, marginTop:2 }
```

```
              </div>
              <span style={{ fontSize:10, color:p.color, border:`1px solid
            </div>
            <p style={{ fontSize:12, color:D.textMuted, lineHeight:1.5, mar
            <div style={{ background:`${p.color}10`, border:`1px solid ${p.
              <span style={{ fontSize:12, fontFamily:D.heading, color:p.col
              <span style={{ color:p.color }}>›</span>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>
)}

{/* ── NUTRITION ── */}
{tab=="nutrition" && (
  <div style={{ padding:"0 16px" }}>
    <div style={{ background:D.redMid, border:`1px solid ${D.borderHi}`, bo
      <div style={{ fontSize:36, marginBottom:8 }}>🥗</div>
      <div style={{ fontSize:20, fontFamily:D.heading, color:D.textPrimary,
      <div style={{ fontSize:12, color:D.textSecondary, lineHeight:1.5 }}>M
    </div>
    {NUTRITION_TIPS.map(n => (
      <div key={n.title} style={{ background:D.bgCard, border:`1px solid ${
        <div style={{ display:"flex", alignItems:"center", gap:10, marginBo
          <span style={{ fontSize:28 }}>{n.icon}</span>
          <div style={{ fontSize:16, fontFamily:D.heading, color:n.color, l
        </div>
        {n.tips.map((t,i) => (
          <div key={i} style={{ display:"flex", gap:8, marginBottom:6, alig
            <span style={{ color:n.color, fontSize:12, flexShrink:0, margin
            <span style={{ fontSize:12, color:D.textSecondary, lineHeight:1
          </div>
        ))}
      </div>
    ))}
  </div>
)}

{/* ── GLOSSAIRE ── */}
{tab=="glossaire" && (
  <div style={{ padding:"0 16px" }}>
    <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, bord
      <span style={{ fontSize:16 }}>🔍</span>
      <input value={glossSearch} onChange={e=>setGlossSearch(e.target.value
    </div>
```

```
          <div style={{ display:"flex", flexDirection:"column", gap:8 }}>
            {GLOSSAIRE.filter(g => !glossSearch || g.fr.toLowerCase().includes(gl
              <div key={i} style={{ background:D.bgCard, border:`1px solid ${D.bo
                <div style={{ display:"flex", justifyContent:"space-between", ali
                  <div>
                    <div style={{ fontSize:20, fontFamily:D.heading, color:D.red,
                    <div style={{ fontSize:16, color:D.textSecondary }}>{g.thai}<
                  </div>
                  <div style={{ background:D.redMid, border:`1px solid ${D.border
                    <div style={{ fontSize:11, fontFamily:D.heading, color:D.red,
                  </div>
                </div>
                <div style={{ fontSize:12, color:D.textMuted, fontStyle:"italic"
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  );
}

function ProgramsScreen({ isPremium, goPaywall }) {
  return <TechniquesScreen isPremium={isPremium} goPaywall={goPaywall} initialTab
}

// ── PROFILE SCREEN ──────────────────────────────────────────────
function ProfileScreen({ setScreen, isPremium, user, onSignOut, trialDaysLeft })
  const progStats = progression.getStats();
  const badgeCount = (JSON.parse(localStorage.getItem("amt_badges_unlocked") || "

  const menuItems = [
    { icon:"📈", label:"MA PROGRESSION", sub:"Séances, calories, streak", screen:
    { icon:"🏆", label:"MES BADGES", sub:`${badgeCount} badge${badgeCount!==1?"s"
    { icon:"⭐", label:"AVIS & NOTES", sub:"Laisser un avis sur l'app", screen:"r
    { icon:"🌟", label:"PARRAINAGE", sub:"Inviter des amis", screen:"referral", c
    { icon:"💬", label:"CHAT COACH IA", sub:"Poser des questions", screen:"chat",
    { icon:"🔔", label:"NOTIFICATIONS", sub:"Rappels d'entraînement", screen:"not
  ];

  return (
    <div style={{ padding:"16px 16px 90px", height:"100%", overflowY:"auto", back

      {/* Avatar */}
      <div style={{ display:"flex", alignItems:"center", gap:14, marginBottom:20,
        <div style={{ width:60, height:60, borderRadius:"50%", background:`linear
        <div style={{ flex:1 }}>
```

```
    <div style={{ fontSize:18, fontFamily:D.heading, color:D.textPrimary, l
      {user?.name || user?.email?.split("@")[0] || "MON PROFIL"}
    </div>
    <div style={{ fontSize:11, color:D.textMuted, marginTop:1 }}>{user?.ema
    {isPremium && trialDaysLeft > 0 && <div style={{ fontSize:11, color:D.g
    {isPremium && trialDaysLeft === 0 && <div style={{ fontSize:11, color:D
    {!isPremium && <div style={{ fontSize:11, color:D.textMuted, marginTop:
  </div>
  <button onClick={onSignOut} style={{ background:D.bgCard, border:`1px sol
</div>

{/* Stats rapides */}
<div style={{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:8, mar
  {[
    { icon:"🥊", value:progStats.sessions||0, label:"Séances" },
    { icon:"🔥", value:`${progStats.streak||0}j`, label:"Streak" },
    { icon:"💪", value:progStats.totalCalories||0, label:"Calories" },
  ].map(s => (
    <div key={s.label} style={{ background:D.bgCard, border:`1px solid ${D.
      <div style={{ fontSize:18, marginBottom:4 }}>{s.icon}</div>
      <div style={{ fontSize:18, fontFamily:D.heading, color:D.textPrimary
      <div style={{ fontSize:9, color:D.textMuted, marginTop:2 }}>{s.label}
    </div>
  ))}
</div>

{/* Premium banner */}
{isPremium ? (
  <div style={{ background:D.goldSoft, border:"1px solid rgba(212,160,23,0.
    <span style={{ fontSize:24 }}>⭐</span>
    <div>
      <div style={{ fontSize:14, fontFamily:D.heading, color:D.gold, letter
      <div style={{ fontSize:11, color:D.textMuted, marginTop:2 }}>Toutes l
    </div>
  </div>
) : (
  <div style={{ background:D.goldSoft, border:"1px solid rgba(212,160,23,0.
    <div style={{ fontSize:13, fontFamily:D.heading, color:D.gold, letterSp
    <div style={{ fontSize:11, color:D.textMuted, marginBottom:12, lineHeig
    <button onClick={() => setScreen("paywall")} style={{ width:"100%", bor
      ⭐ PASSER PREMIUM — 9.99€/mois
    </button>
  </div>
)}

{/* Menu items */}
<div style={{ display:"flex", flexDirection:"column", gap:8, marginBottom:1
```

```
        {menuItems.map(item => (
          <button key={item.screen} onClick={() => setScreen(item.screen)} style=
            <div style={{ width:38, height:38, borderRadius:D.rm, background:`rgb
            <div style={{ flex:1 }}>
              <div style={{ fontSize:13, fontFamily:D.heading, color:D.textPrimar
              <div style={{ fontSize:10, color:D.textMuted, marginTop:2 }}>{item.
            </div>
            <span style={{ color:D.textMuted, fontSize:16 }}>›</span>
          </button>
        ))}
      </div>

      {/* Admin */}
      {user?.email === ADMIN_EMAIL && (
        <button onClick={() => setScreen("admin")} style={{ width:"100%", border:
          <span style={{ fontSize:18 }}>👑</span> PANNEAU COACH ADMIN
        </button>
      )}

      {/* Liens légaux */}
      <div style={{ display:"flex", gap:8, justifyContent:"center", flexWrap:"wra
        <button onClick={() => setScreen("legal-privacy")} style={{ background:"n
        <button onClick={() => setScreen("legal-terms")} style={{ background:"non
      </div>
      <div style={{ textAlign:"center", marginTop:10, fontSize:9, color:D.textMut
    </div>
  );
}

// ── NOTIFICATIONS SYSTEM ──────────────────────────────────────
const NOTIF_MESSAGES = [
  "C'est l'heure de t'entraîner ! 🥊",
  "Rappel d'entraînement 💪",
  "Autam Muay Thai t'attend ! 🔥",
  "Guerrier, c'est l'heure ! 🥊",
];

const notifStorage = {
  get: () => { try{return JSON.parse(localStorage.getItem("amt_notif_settings")||
  set: (d) => localStorage.setItem("amt_notif_settings", JSON.stringify(d)),
};

function NotificationsScreen({setScreen}) {
  const [s, setS] = useState(() => ({
    dailyReminder:true, dailyHour:12, customDays:[1,2,3,4,5],
    inactivityAlert:true, inactivityDays:2, permission:"default",
    ...notifStorage.get(),
```

```
}));
const [testSent, setTestSent] = useState(false);
const [permErr, setPermErr] = useState("");
const save = u => { const n={...s,...u}; setS(n); notifStorage.set(n); };
const tog = on => ({width:44,height:24,borderRadius:12,background:on?D.red:"rgb
const dot = on => ({position:"absolute",width:18,height:18,borderRadius:"50%",b
const days = [{id:0,l:"D"},{id:1,l:"L"},{id:2,l:"M"},{id:3,l:"M"},{id:4,l:"J"},

const reqPerm = async () => {
  setPermErr("");
  if(!("Notification" in window)){setPermErr("Non supporté sur ce navigateur.")
  const p = await Notification.requestPermission();
  save({permission:p});
  if(p==="denied") setPermErr("Bloquées — active-les dans les réglages.");
};
const testNotif = () => {
  if(s.permission!=="granted"){reqPerm();return;}
  new Notification("C'est l'heure de t'entraîner ! 🥊", {body:"Forge ton corps.
  setTestSent(true); setTimeout(()=>setTestSent(false),3000);
};

return (
  <div style={{height:"100%",display:"flex",flexDirection:"column",background:D
    <div style={{background:D.bg,padding:"20px 16px 16px",borderBottom:`1px sol
      <button onClick={()=>setScreen("profile")} style={{background:D.bgCard,bo
      <div>
        <div style={{fontSize:20,fontFamily:D.heading,color:D.textPrimary,lette
        <div style={{fontSize:10,color:D.textMuted}}>Rappels personnalisés</div
      </div>
    </div>
    <div style={{flex:1,overflowY:"auto",padding:16}}>
      {s.permission!=="granted" ? (
        <div style={{background:"rgba(255,26,26,0.06)",border:`1px solid ${D.bo
          <div style={{fontSize:13,fontFamily:D.heading,color:D.red,marginBotto
          <button onClick={reqPerm} style={{width:"100%",border:"none",borderRa
          {permErr && <div style={{fontSize:11,color:D.gold,marginTop:8}}>⚠ {pe
        </div>
      ) : (
        <div style={{background:D.greenSoft,border:"1px solid rgba(0,230,118,0.
          <span style={{fontSize:16}}>✅</span><span style={{fontSize:12,color:
        </div>
      )}

      <div style={{background:D.bgCard,border:`1px solid ${D.border}`,borderRad
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"
          <div>
            <div style={{fontSize:14,fontFamily:D.heading,color:D.textPrimary,l
```

```
            <div style={{fontSize:11,color:D.textMuted,marginTop:2}}>Un rappel
          </div>
          <button onClick={()=>save({dailyReminder:!s.dailyReminder})} style={t
        </div>
        {s.dailyReminder && (
          <div style={{display:"flex",gap:6,flexWrap:"wrap"}}>
            {[7,8,9,12,13,17,18,19,20,21].map(h=>(
              <button key={h} onClick={()=>save({dailyHour:h})} style={{backgro
            ))}
          </div>
        )}
      </div>

      <div style={{background:D.bgCard,border:`1px solid ${D.border}`,borderRad
        <div style={{fontSize:14,fontFamily:D.heading,color:D.textPrimary,lette
        <div style={{display:"flex",gap:6,justifyContent:"space-between"}}>
          {days.map(d=>(
            <button key={d.id} onClick={()=>{const ds=s.customDays.includes(d.i
          ))}
        </div>
      </div>

      <div style={{background:D.bgCard,border:`1px solid ${D.border}`,borderRad
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"
          <div>
            <div style={{fontSize:14,fontFamily:D.heading,color:D.textPrimary,l
            <div style={{fontSize:11,color:D.textMuted,marginTop:2}}>Si tu n'ou
          </div>
          <button onClick={()=>save({inactivityAlert:!s.inactivityAlert})} styl
        </div>
        {s.inactivityAlert && (
          <div style={{display:"flex",gap:8}}>
            {[1,2,3,5,7].map(d=>(
              <button key={d} onClick={()=>save({inactivityDays:d})} style={{ba
            ))}
          </div>
        )}
      </div>

      <button onClick={testNotif} style={{width:"100%",border:`1px solid ${test
        {testSent?"✅ ENVOYÉE !":"🔔 TESTER UNE NOTIFICATION"}
      </button>
      <div style={{textAlign:"center",fontSize:10,color:D.textMuted,lineHeight:
    </div>
  </div>
);
}
```

```
// ── LEGAL SCREEN ──────────────────────────────────────────────
function LegalScreen({ setScreen, initialTab = "privacy" }) {
  const [tab, setTab] = useState(initialTab);

  const privacy = [
    { title: "Qui sommes-nous ?", content: "Autam Muay Thai est une application m
    { title: "Données collectées", content: "Nous collectons : votre email et nom
    { title: "Utilisation des données", content: "Vos données servent uniquement
    { title: "Partenaires", content: "Stripe (paiements), Supabase (base de donné
    { title: "Vos droits (RGPD)", content: "Vous pouvez à tout moment : accéder à
    { title: "Sécurité", content: "Vos données sont chiffrées SSL/TLS, mots de pa
    { title: "Enfants", content: "L'application est destinée aux personnes de 13
  ];

  const terms = [
    { title: "Acceptation", content: "En utilisant Autam Muay Thai, vous acceptez
    { title: "Essai gratuit", content: "7 jours Premium gratuit à l'inscription,
    { title: "Abonnement Premium", content: "9.99€/mois ou 99.99€/an, renouvelé a
    { title: "Utilisations interdites", content: "Il est interdit de : revendre l
    { title: "⚠️ Avertissement santé", content: "Le Muay Thai comporte des risque
    { title: "Propriété intellectuelle", content: "L'app, son logo, ses contenus
    { title: "Responsabilité", content: "Autam Muay Thai n'est pas responsable de
    { title: "Contact", content: "support@autammuaythai.com — Réponse sous 48h ou
  ];

  const items = tab === "privacy" ? privacy : terms;

  return (
    <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
      {/* Header */}
      <div style={{background: "linear-gradient(135deg, #0a0a0a, #1a0000)", paddi
        <button onClick={() => setScreen("profile")} style={{background: "rgba(25
        <div>
          <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color:
          <div style={{ fontSize: 10, color: "#666" }}>Autam Muay Thai · Version
        </div>
      </div>

      {/* Tabs */}
      <div style={{ display: "flex", background: "rgba(255,255,255,0.03)", border
        {[
          { id: "privacy", label: "🔒 Confidentialité" },
          { id: "terms", label: "📋 CGU" },
        ].map(t => (
          <button key={t.id} onClick={() => setTab(t.id)} style={{flex: 1, backgr
        ))}
```

```
      </div>

      {/* Content */}
      <div style={{ flex: 1, overflowY: "auto", padding: "16px" }}>
        {items.map((item, i) => (
          <div key={i} style={{background: "rgba(255,255,255,0.03)", border: "1px
            <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", colo
              {item.title}
            </div>
            <div style={{ fontSize: 12, color: "#aaa", lineHeight: 1.7 }}>{item.c
          </div>
        ))}

        <div style={{ textAlign: "center", padding: "20px 0", borderTop: "1px sol
          <div style={{ fontSize: 24, marginBottom: 6 }}>🥊</div>
          <div style={{ fontSize: 10, color: "#444" }}>Autam Muay Thai · contact@
        </div>
      </div>
    </div>
  );
}

// ── STRIPE LINKS ──────────────────────────────────────────────
const STRIPE_MONTHLY = "https://buy.stripe.com/4gM3cugClel31oxgPB18c00";
const STRIPE_YEARLY  = "https://buy.stripe.com/4gMdR80Dnel39V3fLx18c01";

// ── PAYWALL SCREEN ────────────────────────────────────────────
function PaywallScreen({ setScreen, onActivate }) {
  const [plan, setPlan] = useState("monthly");

  const freeFeatures = [
    "1 analyse Coach IA par jour",
    "Timer basique",
    "3 combos prédéfinis",
    "Techniques de base",
  ];

  const premiumFeatures = [
    "Coach IA illimité",
    "Créer ses propres combos",
    "Vidéos de référence du coach",
    "Historique complet + graphiques",
    "Programmes personnalisés par l'IA",
    "Mode live illimité",
    "Badges et progression",
    "Toutes les techniques avancées",
  ];
```

```
const handleSubscribe = () => {
  const url = plan === "monthly" ? STRIPE_MONTHLY : STRIPE_YEARLY;
  window.open(url, "_blank");
  setTimeout(() => { onActivate(); setScreen("home"); }, 1000);
};

return (
  <div style={{ padding: "0 0 80px", height: "100%", overflowY: "auto" }}>
    {/* Header */}
    <div style={{background: "linear-gradient(135deg, #1a0a00, #0a0a0a)", paddi
      <button onClick={() => setScreen("profile")} style={{position: "absolute"
      <div style={{ fontSize: 36, marginBottom: 8 }}>⭐</div>
      <div style={{ fontSize: 32, fontFamily: "'Bebas Neue', cursive", color: "
      <div style={{ fontSize: 13, color: "#888", marginTop: 6, fontStyle: "ital
        Débloque tout le potentiel de ton entraînement
      </div>
    </div>

    <div style={{ padding: "20px 16px 0" }}>
      {/* Comparaison */}
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10, m
        {/* Gratuit */}
        <div style={{ background: "rgba(255,255,255,0.03)", border: "1px solid
          <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive', curs
          {freeFeatures.map((f, i) => (
            <div key={i} style={{ display: "flex", gap: 8, marginBottom: 8, ali
              <span style={{ color: "#22c55e", fontSize: 12, marginTop: 1 }}>✓<
              <span style={{ fontSize: 11, color: "#888", lineHeight: 1.4 }}>{f
            </div>
          ))}
        </div>
        {/* Premium */}
        <div style={{ background: "rgba(245,158,11,0.08)", border: "1px solid r
          <div style={{ position: "absolute", top: -12, right: -12, fontSize: 6
          <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", colo
          {premiumFeatures.map((f, i) => (
            <div key={i} style={{ display: "flex", gap: 8, marginBottom: 8, ali
              <span style={{ color: "#f59e0b", fontSize: 12, marginTop: 1 }}>⭐
              <span style={{ fontSize: 11, color: "#ddd", lineHeight: 1.4 }}>{f
            </div>
          ))}
        </div>
      </div>

      {/* Sélecteur de plan */}
      <div style={{ fontSize: 11, color: "#f59e0b", letterSpacing: 3, fontFamil
```

```
      <div style={{ display: "flex", flexDirection: "column", gap: 10, marginBo
        {/* Mensuel */}
        <div onClick={() => setPlan("monthly")} style={{background: plan === "m
          <div>
            <div style={{ fontSize: 15, fontFamily: "'Bebas Neue', cursive", co
            <div style={{ fontSize: 11, color: "#666", marginTop: 2 }}>Sans eng
          </div>
          <div style={{ textAlign: "right" }}>
            <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", co
            <div style={{ fontSize: 10, color: "#666" }}>/mois</div>
          </div>
        </div>

        {/* Annuel */}
        <div onClick={() => setPlan("yearly")} style={{background: plan === "ye
          <div style={{position: "absolute", top: 0, right: 0, background: "#ef
          <div>
            <div style={{ fontSize: 15, fontFamily: "'Bebas Neue', cursive", co
            <div style={{ fontSize: 11, color: "#666", marginTop: 2 }}>99.99€/a
          </div>
          <div style={{ textAlign: "right" }}>
            <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", co
            <div style={{ fontSize: 10, color: "#666" }}>/mois</div>
          </div>
        </div>
      </div>

      {/* CTA */}
      <button onClick={handleSubscribe} style={{width: "100%", border: "none",
        {`⭐ PASSER PREMIUM — ${plan === "monthly" ? "9.99€/mois" : "99.99€/an"
      </button>

      {/* Stripe badge */}
      <div style={{ display: "flex", alignItems: "center", justifyContent: "cen
        <span style={{ fontSize: 14 }}>🔒</span>
        <span style={{ fontSize: 11, color: "#f59e0b", fontFamily: "'Bebas Neue
          PAIEMENT SÉCURISÉ VIA STRIPE
        </span>
      </div>

      <div style={{ textAlign: "center", fontSize: 10, color: "#555", lineHeigh
        Tu seras redirigé vers la page de paiement Stripe<br/>
        Annulation à tout moment · Aucun frais caché<br/>
        Satisfait ou remboursé 7 jours
      </div>
    </div>
  </div>
```

```
  );
}

// ── PREMIUM LOCK OVERLAY ──────────────────────────────────────
function PremiumLock({ feature, onGoPaywall }) {
  return (
    <div onClick={onGoPaywall} style={{position: "absolute", inset: 0, zIndex: 20
      <div style={{ fontSize: 36, marginBottom: 10 }}>🔒</div>
      <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color: "#f
        FONCTIONNALITÉ PREMIUM
      </div>
      <div style={{ fontSize: 12, color: "#888", marginBottom: 16, lineHeight: 1.
      <div style={{background: "linear-gradient(135deg, #f59e0b, #d97706)", color
    </div>
  );
}

// ── ADMIN CONFIG ──────────────────────────────────────────────
const ADMIN_EMAIL = "mathispozner1@gmail.com";
const isAdmin = (user) => user?.email === ADMIN_EMAIL;

// ── ADMIN SCREEN ──────────────────────────────────────────────
function AdminScreen({ setScreen }) {
  const [tab, setTab] = useState("combos"); // combos | techniques | users
  const [combos, setCombos] = useState(() => {
    try { return JSON.parse(localStorage.getItem("admin_combos") || "[]"); } catc
  });
  const [techniques, setTechniques] = useState(() => {
    try { return JSON.parse(localStorage.getItem("admin_techniques") || "[]"); }
  });
  const [users] = useState(() => {
    try { return Object.values(JSON.parse(localStorage.getItem("amt_users") || "{
  });

  const [showComboForm, setShowComboForm] = useState(false);
  const [editCombo, setEditCombo] = useState(null);
  const [comboName, setComboName] = useState("");
  const [comboSteps, setComboSteps] = useState("");
  const [comboCues, setComboCues] = useState("");
  const [comboLevel, setComboLevel] = useState("Débutant");
  const [comboVideo, setComboVideo] = useState(null);
  const [comboVideoPreview, setComboVideoPreview] = useState(null);
  const comboVideoRef = useRef(null);

  const [showTechForm, setShowTechForm] = useState(false);
  const [editTech, setEditTech] = useState(null);
  const [techName, setTechName] = useState("");
```

```
const [techCategory, setTechCategory] = useState("Poing");
const [techEmoji, setTechEmoji] = useState("");
const [techDesc, setTechDesc] = useState("");
const [techVideo, setTechVideo] = useState(null);
const [techVideoPreview, setTechVideoPreview] = useState(null);
const techVideoRef = useRef(null);

const [saved, setSaved] = useState(false);

const saveCombo = () => {
  if (!comboName.trim()) return;
  const steps = comboSteps.split("\n").map(s => s.trim()).filter(Boolean);
  const cues = comboCues.split("\n").map(s => s.trim()).filter(Boolean);
  const newCombo = {
    id: editCombo?.id || Date.now().toString(),
    name: comboName, steps, cues, level: comboLevel,
    refVideoUrl: comboVideoPreview || editCombo?.refVideoUrl || null,
    createdAt: editCombo?.createdAt || Date.now(),
  };
  const updated = editCombo
    ? combos.map(c => c.id === editCombo.id ? newCombo : c)
    : [...combos, newCombo];
  setCombos(updated);
  localStorage.setItem("admin_combos", JSON.stringify(updated));
  setShowComboForm(false); setEditCombo(null);
  setComboName(""); setComboSteps(""); setComboCues(""); setComboVideoPreview(n
  showSaved();
};

const deleteCombo = (id) => {
  const updated = combos.filter(c => c.id !== id);
  setCombos(updated);
  localStorage.setItem("admin_combos", JSON.stringify(updated));
};

const saveTech = () => {
  if (!techName.trim()) return;
  const newTech = {
    id: editTech?.id || Date.now().toString(),
    name: techName, category: techCategory, emoji: techEmoji, desc: techDesc,
    videoUrl: techVideoPreview || editTech?.videoUrl || null,
    createdAt: editTech?.createdAt || Date.now(),
  };
  const updated = editTech
    ? techniques.map(t => t.id === editTech.id ? newTech : t)
    : [...techniques, newTech];
  setTechniques(updated);
```

```
  localStorage.setItem("admin_techniques", JSON.stringify(updated));
  setShowTechForm(false); setEditTech(null);
  setTechName(""); setTechDesc(""); setTechVideoPreview(null);
  showSaved();
};

const deleteTech = (id) => {
  const updated = techniques.filter(t => t.id !== id);
  setTechniques(updated);
  localStorage.setItem("admin_techniques", JSON.stringify(updated));
};

const showSaved = () => { setSaved(true); setTimeout(() => setSaved(false), 200

const S = {
  input: { width: "100%", background: "rgba(255,255,255,0.07)", border: "1px so
  btn: (color) => ({ background: color, border: "none", color: "#fff", borderRa
};

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    {/* Header */}
    <div style={{ background: "linear-gradient(135deg, #0a0000, #1a0a00)", padd
      <button onClick={() => setScreen("profile")} style={{ background: "rgba(2
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 10, color: "#666" }}>Accès exclusif · mathispoz
      </div>
      {saved && <div style={{ fontSize: 11, color: "#22c55e", fontFamily: "'Beb
    </div>

    {/* Tabs */}
    <div style={{ display: "flex", background: "rgba(255,255,255,0.03)", border
      {[
        { id: "combos", label: "🥊 Combos", count: combos.length },
        { id: "techniques", label: "⚡ Techniques", count: techniques.length },
        { id: "users", label: "👥 Membres", count: users.length },
      ].map(t => (
        <button key={t.id} onClick={() => setTab(t.id)} style={{flex: 1, backgr
      ))}
    </div>

    <div style={{ flex: 1, overflowY: "auto", padding: 14 }}>

      {/* ── TAB COMBOS ── */}
      {tab === "combos" && (
        <div>
```

```
<div style={{ display: "flex", justifyContent: "space-between", align
  <div style={{ fontSize: 11, color: "#f59e0b", fontFamily: "'Bebas N
    GÉRER LES COMBOS ({combos.length})
  </div>
  <button onClick={() => { setShowComboForm(true); setEditCombo(null)
    + NOUVEAU
  </button>
</div>

{showComboForm && (
  <div style={{ background: "rgba(245,158,11,0.08)", border: "1px sol
    <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive",
      {editCombo ? "MODIFIER LE COMBO" : "NOUVEAU COMBO"}
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 10 }
      <input placeholder="Nom du combo" value={comboName} onChange={e
      <div>
        <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}
        <select value={comboLevel} onChange={e => setComboLevel(e.tar
          {["Débutant", "Intermédiaire", "Avancé"].map(l => <option k
        </select>
      </div>
      <div>
        <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}
        <textarea placeholder={"Jab\nCross\nRight Middle Kick"} value
      </div>
      <div>
        <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}
        <textarea placeholder={"Garde haute\nRotation des hanches"} v
      </div>
      <div>
        <div style={{ fontSize: 10, color: "#f59e0b", marginBottom: 6
        <input ref={comboVideoRef} type="file" accept="video/*,image/
          const f = e.target.files[0]; if (!f) return;
          setComboVideo(f); setComboVideoPreview(URL.createObjectURL(
        }} />
        <div onClick={() => comboVideoRef.current?.click()} style={{
          {comboVideoPreview ? (
            <video src={comboVideoPreview} controls style={{ width: "
          ) : (
            <div style={{ fontSize: 11, color: "#666" }}>📁 Appuie pc
          )}
        </div>
      </div>
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr",
        <button onClick={() => setShowComboForm(false)} style={S.btn(
        <button onClick={saveCombo} style={S.btn("#f59e0b")}>💾 SAUVE
```

```
          </div>
        </div>
      </div>
    )}

    {combos.length === 0 ? (
      <div style={{ textAlign: "center", padding: 32, color: "#444", font
        Aucun combo ajouté.<br/>Crée ton premier combo !
      </div>
    ) : (
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
        {combos.map(c => (
          <div key={c.id} style={{ background: "rgba(255,255,255,0.03)",
            <div style={{ display: "flex", justifyContent: "space-between
              <div>
                <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cu
                <div style={{ fontSize: 10, color: "#f59e0b", marginTop:
                <div style={{ fontSize: 10, color: "#666", marginTop: 2 }
              </div>
              <div style={{ display: "flex", gap: 6 }}>
                <button onClick={() => { setEditCombo(c); setComboName(c.
                <button onClick={() => deleteCombo(c.id)} style={{ backgr
              </div>
            </div>
            {c.refVideoUrl && <div style={{ fontSize: 10, color: "#22c55e
          </div>
        ))}
      </div>
    )}
  </div>
)}

{/* ── TAB TECHNIQUES ── */}
{tab === "techniques" && (
  <div>
    <div style={{ display: "flex", justifyContent: "space-between", align
      <div style={{ fontSize: 11, color: "#f59e0b", fontFamily: "'Bebas N
        GÉRER LES TECHNIQUES ({techniques.length})
      </div>
      <button onClick={() => { setShowTechForm(true); setEditTech(null);
        + NOUVELLE
      </button>
    </div>

    {showTechForm && (
      <div style={{ background: "rgba(245,158,11,0.08)", border: "1px sol
        <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive",
```

```
        {editTech ? "MODIFIER LA TECHNIQUE" : "NOUVELLE TECHNIQUE"}
      </div>
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }
        <input placeholder="Nom de la technique" value={techName} onCha
        <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr",
          <div>
            <div style={{ fontSize: 10, color: "#666", marginBottom: 4
            <select value={techCategory} onChange={e => setTechCategory
              {["Poing", "Coup de pied", "Coude", "Genou", "Corps à cor
            </select>
          </div>
          <div>
            <div style={{ fontSize: 10, color: "#666", marginBottom: 4
            <input placeholder="🥊" value={techEmoji} onChange={e => se
          </div>
        </div>
        <div>
          <div style={{ fontSize: 10, color: "#666", marginBottom: 4 }}
          <textarea placeholder="Décris la technique..." value={techDes
        </div>
        <div>
          <div style={{ fontSize: 10, color: "#f59e0b", marginBottom: 6
          <input ref={techVideoRef} type="file" accept="video/*,image/*
            const f = e.target.files[0]; if (!f) return;
            setTechVideo(f); setTechVideoPreview(URL.createObjectURL(f)
          }} />
          <div onClick={() => techVideoRef.current?.click()} style={{ b
            {techVideoPreview ? (
              <video src={techVideoPreview} controls style={{ width: "1
            ) : (
              <div style={{ fontSize: 11, color: "#666" }}>📁 Appuie pc
            )}
          </div>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr",
          <button onClick={() => setShowTechForm(false)} style={S.btn("
          <button onClick={saveTech} style={S.btn("#f59e0b")}>💾 SAUVEG
        </div>
      </div>
    </div>
  )}

  {techniques.length === 0 ? (
    <div style={{ textAlign: "center", padding: 32, color: "#444", font
      Aucune technique ajoutée.<br/>Crée ta première technique !
    </div>
  ) : (
```

```
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
        {techniques.map(t => (
          <div key={t.id} style={{ background: "rgba(255,255,255,0.03)",
            <div style={{ display: "flex", justifyContent: "space-between
              <div style={{ display: "flex", gap: 10, alignItems: "center
                <span style={{ fontSize: 24 }}>{t.emoji}</span>
                <div>
                  <div style={{ fontSize: 13, fontFamily: "'Bebas Neue',
                  <div style={{ fontSize: 10, color: "#ef4444" }}>{t.cate
                </div>
              </div>
              <div style={{ display: "flex", gap: 6 }}>
                <button onClick={() => { setEditTech(t); setTechName(t.na
                <button onClick={() => deleteTech(t.id)} style={{ backgro
              </div>
            </div>
            {t.videoUrl && <div style={{ fontSize: 10, color: "#22c55e",
          </div>
        ))}
      </div>
    )}
  </div>
)}

{/* ── TAB USERS ── */}
{tab === "users" && (
  <div>
    <div style={{ fontSize: 11, color: "#f59e0b", fontFamily: "'Bebas Neu
      MEMBRES INSCRITS ({users.length})
    </div>

    {/* Stats rapides */}
    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8
      {[
        { label: "Total membres", value: users.length, icon: "👥" },
        { label: "Inscrits aujourd'hui", value: users.filter(u => new Dat
      ].map(s => (
        <div key={s.label} style={{ background: "rgba(245,158,11,0.08)",
          <div style={{ fontSize: 20, marginBottom: 4 }}>{s.icon}</div>
          <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive"
          <div style={{ fontSize: 10, color: "#666" }}>{s.label}</div>
        </div>
      ))}
    </div>

    {users.length === 0 ? (
      <div style={{ textAlign: "center", padding: 32, color: "#444", font
```

```
                Aucun membre inscrit pour l'instant.
              </div>
            ) : (
              <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
                {users.map((u, i) => (
                  <div key={i} style={{ background: "rgba(255,255,255,0.03)", bor
                    <div style={{ width: 36, height: 36, borderRadius: "50%", bac
                    <div style={{ flex: 1 }}>
                      <div style={{ fontSize: 13, color: "#fff" }}>{u.name || "Sa
                      <div style={{ fontSize: 10, color: "#666" }}>{u.email}</div
                      <div style={{ fontSize: 10, color: "#444", marginTop: 2 }}>
                        Inscrit le {new Date(u.createdAt).toLocaleDateString("fr-
                      </div>
                    </div>
                    {u.email === ADMIN_EMAIL && (
                      <span style={{ fontSize: 10, color: "#f59e0b", fontFamily:
                    )}
                  </div>
                ))}
              </div>
            )}
          </div>
        )}
      </div>
    </div>
  );
}

// ── REVIEW SYSTEM ──────────────────────────────────────────────
const reviewStorage = {
  get: () => { try { return JSON.parse(localStorage.getItem("amt_reviews") || "[]
  add: (r) => {
    const reviews = reviewStorage.get();
    reviews.unshift({ ...r, id: Date.now(), date: new Date().toLocaleDateString("
    localStorage.setItem("amt_reviews", JSON.stringify(reviews.slice(0, 100)));
    return reviews;
  },
  getMyReview: (userId) => {
    const reviews = reviewStorage.get();
    return reviews.find(r => r.userId === userId);
  },
  getStats: () => {
    const reviews = reviewStorage.get();
    if (!reviews.length) return { avg: 0, total: 0, dist: [0,0,0,0,0] };
    const dist = [0,0,0,0,0];
    reviews.forEach(r => { if(r.rating >= 1 && r.rating <= 5) dist[r.rating-1]++;
    const avg = reviews.reduce((s,r) => s + r.rating, 0) / reviews.length;
```

```
    return { avg: Math.round(avg * 10) / 10, total: reviews.length, dist };
  },
};

if (!localStorage.getItem("amt_reviews_seeded")) {
  const demoReviews = [
    { id:1, userId:"demo1", name:"Alex M.", level:"Intermédiaire", rating:5, comm
    { id:2, userId:"demo2", name:"Sarah K.", level:"Débutant", rating:5, comment:
    { id:3, userId:"demo3", name:"Marco R.", level:"Avancé", rating:4, comment:"T
    { id:4, userId:"demo4", name:"Léa D.", level:"Débutant", rating:5, comment:"L
    { id:5, userId:"demo5", name:"Thomas B.", level:"Intermédiaire", rating:4, co
  ];
  localStorage.setItem("amt_reviews", JSON.stringify(demoReviews));
  localStorage.setItem("amt_reviews_seeded", "1");
}

function StarRating({ value, onChange, size = 32 }) {
  const [hover, setHover] = useState(0);
  return (
    <div style={{ display:"flex", gap:6 }}>
      {[1,2,3,4,5].map(n => (
        <span key={n}
          onClick={() => onChange && onChange(n)}
          onMouseEnter={() => onChange && setHover(n)}
          onMouseLeave={() => onChange && setHover(0)}
          style={{ fontSize:size, cursor:onChange?"pointer":"default", transition
          ⭐
        </span>
      ))}
    </div>
  );
}

function ReviewsScreen({ setScreen, user }) {
  const [tab, setTab] = useState("reviews"); // reviews | write
  const [rating, setRating] = useState(0);
  const [comment, setComment] = useState("");
  const [level, setLevel] = useState("Débutant");
  const [submitted, setSubmitted] = useState(false);
  const [reviews, setReviews] = useState(reviewStorage.get());
  const stats = reviewStorage.getStats();
  const myReview = reviewStorage.getMyReview(user?.id);

  const submitReview = () => {
    if (!rating || !comment.trim()) return;
    reviewStorage.add({
      userId: user?.id,
```

```
    name: user?.name || user?.email?.split("@")[0] || "Anonyme",
    level, rating, comment: comment.trim(),
  });
  setReviews(reviewStorage.get());
  setSubmitted(true);
  setTab("reviews");
};

const levelColors = { "Débutant":"#22c55e", "Intermédiaire":"#f59e0b", "Avancé"

return (
  <div style={{ height:"100%", display:"flex", flexDirection:"column", backgrou
    {/* Header */}
    <div style={{ background:D.bg, padding:"20px 16px 16px", borderBottom:`1px
      <button onClick={() => setScreen("profile")} style={{ background:D.bgCard
      <div style={{ flex:1 }}>
        <div style={{ fontSize:20, fontFamily:D.heading, color:D.textPrimary, l
        <div style={{ fontSize:10, color:D.textMuted }}>{stats.total} avis · {s
      </div>
      {!myReview && (
        <button onClick={() => setTab(tab==="write"?"reviews":"write")} style={
          {tab==="write" ? "VOIR AVIS" : "✍️ NOTER"}
        </button>
      )}
    </div>

    <div style={{ flex:1, overflowY:"auto", padding:16 }}>

      {/* Stats globales */}
      <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
        <div style={{ display:"flex", alignItems:"center", gap:20, marginBottom
          <div style={{ textAlign:"center" }}>
            <div style={{ fontSize:52, fontFamily:D.heading, color:D.textPrimar
            <StarRating value={Math.round(stats.avg)} size={20} />
            <div style={{ fontSize:11, color:D.textMuted, marginTop:4 }}>{stats
          </div>
          <div style={{ flex:1 }}>
            {[5,4,3,2,1].map(n => {
              const count = stats.dist[n-1] || 0;
              const pct = stats.total ? (count/stats.total)*100 : 0;
              return (
                <div key={n} style={{ display:"flex", alignItems:"center", gap:
                  <span style={{ fontSize:11, color:D.textMuted, width:8 }}>{n}
                  <span style={{ fontSize:12 }}>⭐</span>
                  <div style={{ flex:1, height:6, background:"rgba(255,255,255,
                    <div style={{ height:"100%", width:`${pct}%`, background:`l
                  </div>
```

```
            <span style={{ fontSize:11, color:D.textMuted, width:16, text
          </div>
        );
      })}
    </div>
  </div>
  {submitted && (
    <div style={{ background:D.greenSoft, border:"1px solid rgba(0,230,11
      <span style={{ fontSize:16 }}>✅</span>
      <span style={{ fontSize:12, color:D.green, fontFamily:D.heading, le
    </div>
  )}
</div>

{/* Formulaire d'avis */}
{tab=="write" && !myReview && (
  <div style={{ background:D.bgCard, border:`1px solid ${D.borderHi}`, bo
    <SLabel>✍️ LAISSER TON AVIS</SLabel>

    {/* Étoiles */}
    <div style={{ marginBottom:14 }}>
      <div style={{ fontSize:11, color:D.textMuted, marginBottom:8 }}>Ta
      <StarRating value={rating} onChange={setRating} size={36} />
      {rating > 0 && (
        <div style={{ fontSize:12, color:D.red, fontFamily:D.heading, let
          {["","😞 Mauvais","😕 Moyen","😊 Bien","😄 Très bien","🔥 Exce
        </div>
      )}
    </div>

    {/* Niveau */}
    <div style={{ marginBottom:14 }}>
      <div style={{ fontSize:11, color:D.textMuted, marginBottom:8 }}>Ton
      <div style={{ display:"flex", gap:8 }}>
        {["Débutant","Intermédiaire","Avancé"].map(l => (
          <button key={l} onClick={() => setLevel(l)} style={{flex:1, bac
        ))}
      </div>
    </div>

    {/* Commentaire */}
    <div style={{ marginBottom:14 }}>
      <div style={{ fontSize:11, color:D.textMuted, marginBottom:8 }}>Ton
      <textarea
        value={comment} onChange={e => setComment(e.target.value)}
        placeholder="Partage ton expérience avec Autam Muay Thai..."
        style={{ width:"100%", background:"rgba(255,255,255,0.05)", borde
```

```
              />
              <div style={{ fontSize:10, color:D.textMuted, textAlign:"right", ma
            </div>

            <button onClick={submitReview} disabled={!rating || !comment.trim()}
              ⭐ PUBLIER MON AVIS
            </button>
          </div>
        )}

        {myReview && (
          <div style={{ background:D.redMid, border:`1px solid ${D.borderHi}`, bo
            <span style={{ fontSize:20 }}>✅</span>
            <div>
              <div style={{ fontSize:13, fontFamily:D.heading, color:D.red, lette
              <div style={{ fontSize:11, color:D.textMuted, marginTop:2 }}>Merci
            </div>
          </div>
        )}

        {/* Liste des avis */}
        <SLabel>TOUS LES AVIS</SLabel>
        <div style={{ display:"flex", flexDirection:"column", gap:10 }}>
          {reviews.map(r => (
            <div key={r.id} style={{ background:D.bgCard, border:`1px solid ${D.b
              <div style={{ display:"flex", justifyContent:"space-between", align
                <div style={{ display:"flex", alignItems:"center", gap:10 }}>
                  <div style={{ width:36, height:36, borderRadius:"50%", backgrou
                  <div>
                    <div style={{ fontSize:13, fontFamily:D.heading, color:D.text
                    <div style={{ display:"flex", alignItems:"center", gap:6, mar
                      <span style={{ fontSize:9, color:levelColors[r.level]||D.re
                      <span style={{ fontSize:10, color:D.textMuted }}>{r.date}</
                    </div>
                  </div>
                </div>
                <StarRating value={r.rating} size={14} />
              </div>
              <p style={{ fontSize:13, color:D.textSecondary, lineHeight:1.6, mar
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

```
// ── REFERRAL SYSTEM ──────────────────────────────────────────
const referral = {
  getCode: (user) => {
    if (!user?.email) return null;
    const hash = user.email.split("").reduce((a,c) => ((a<<5)-a)+c.charCodeAt(0),
    return "AUTAM" + Math.abs(hash).toString(36).toUpperCase().slice(0,5);
  },
  getStats: () => {
    try { return JSON.parse(localStorage.getItem("amt_referral") || '{"invited":[
    catch { return { invited:[], rewarded:false }; }
  },
  addInvited: (email) => {
    const s = referral.getStats();
    if (!s.invited.includes(email)) {
      s.invited.push(email);
      localStorage.setItem("amt_referral", JSON.stringify(s));
    }
    return s;
  },
  claimBadge: () => {
    const s = referral.getStats();
    s.rewarded = true;
    localStorage.setItem("amt_referral", JSON.stringify(s));
  },
};

function ReferralScreen({ setScreen, user, isPremium, onBadge }) {
  const code = referral.getCode(user);
  const stats = referral.getStats();
  const [copied, setCopied] = useState(false);
  const [email, setEmail] = useState("");
  const [sent, setSent] = useState(false);
  const [claimed, setClaimed] = useState(stats.rewarded);

  const inviteLink = `https://autammuaythai.com/join?ref=${code}`;
  const inviteText = `🥊 Rejoins-moi sur Autam Muay Thai !\n\nL'app de Muay Thai

  const copyCode = async () => {
    try {
      await navigator.clipboard.writeText(inviteLink);
      setCopied(true);
      setTimeout(() => setCopied(false), 2500);
    } catch {
      setCopied(true);
      setTimeout(() => setCopied(false), 2500);
    }
  };
```

```
const shareInvite = async () => {
  try {
    if (navigator.share) {
      await navigator.share({ title:"Autam Muay Thai", text:inviteText, url:inv
    } else {
      await navigator.clipboard.writeText(inviteText);
      alert("Invitation copiée dans le presse-papiers !");
    }
  } catch {}
};

const sendEmailInvite = () => {
  if (!email.trim()) return;
  referral.addInvited(email);
  setSent(true);
  setEmail("");
  setTimeout(() => setSent(false), 3000);
};

const claimAmbassador = () => {
  if (stats.invited.length >= 1 && !claimed) {
    referral.claimBadge();
    setClaimed(true);
    const ambassadorBadge = { id:"ambassador", icon:"🌟", name:"Ambassadeur", d
    const unlocked = JSON.parse(localStorage.getItem("amt_badges_unlocked") ||
    if (!unlocked.includes("ambassador")) {
      unlocked.push("ambassador");
      localStorage.setItem("amt_badges_unlocked", JSON.stringify(unlocked));
      setTimeout(() => onBadge && onBadge(ambassadorBadge), 500);
    }
  }
};

const progress = Math.min(stats.invited.length, 3);

return (
  <div style={{ height:"100%", display:"flex", flexDirection:"column", backgrou
    {/* Header */}
    <div style={{ background:D.bg, padding:"20px 16px 16px", borderBottom:`1px
      <button onClick={() => setScreen("profile")} style={{ background:D.bgCard
      <div>
        <div style={{ fontSize:20, fontFamily:D.heading, color:D.textPrimary, l
        <div style={{ fontSize:10, color:D.textMuted }}>Invite tes amis · Gagne
      </div>
    </div>
```

```
<div style={{ flex:1, overflowY:"auto", padding:16 }}>

  {/* Hero */}
  <div style={{ background:`linear-gradient(135deg,${D.redDeep}30,${D.redMi
    <div style={{ position:"absolute", right:-20, top:-20, fontSize:80, opa
    <div style={{ fontSize:48, marginBottom:8 }}>🥊</div>
    <div style={{ fontSize:24, fontFamily:D.heading, color:D.textPrimary, l
      INVITE TES AMIS
    </div>
    <div style={{ fontSize:13, color:D.textSecondary, lineHeight:1.6 }}>
      Chaque ami invité reçoit <span style={{ color:D.gold, fontFamily:D.he
      Toi tu gagnes le badge exclusif <span style={{ color:D.gold, fontFami
    </div>
  </div>

  {/* Ton code */}
  <div style={{ marginBottom:16 }}>
    <SLabel>TON CODE DE PARRAINAGE</SLabel>
    <div style={{ background:D.bgCard, border:`1px solid ${D.borderHi}`, bo
      <div>
        <div style={{ fontSize:28, fontFamily:D.heading, color:D.red, lette
        <div style={{ fontSize:11, color:D.textMuted, marginTop:2 }}>Partag
      </div>
      <button onClick={copyCode} style={{ background:copied?D.greenSoft:D.r
        {copied ? "✅ COPIÉ" : "📋 COPIER"}
      </button>
    </div>
  </div>

  {/* Progression ambassadeur */}
  <div style={{ marginBottom:16 }}>
    <SLabel>PROGRESSION AMBASSADEUR</SLabel>
    <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, bord
      <div style={{ display:"flex", justifyContent:"space-between", marginB
        <span style={{ fontSize:12, color:D.textSecondary }}>Amis invités</
        <span style={{ fontSize:12, color:D.gold, fontFamily:D.heading }}>{
      </div>
      <div style={{ height:8, background:"rgba(255,255,255,0.06)", borderRa
        <div style={{ height:"100%", width:`${(progress/3)*100}%`, backgrou
      </div>
      <div style={{ display:"flex", gap:8 }}>
        {[1,2,3].map(n => (
          <div key={n} style={{ flex:1, background:stats.invited.length>=n?
            <div style={{ fontSize:16 }}>{stats.invited.length>=n?"🥊":"o"}
            <div style={{ fontSize:9, color:stats.invited.length>=n?D.red:D
          </div>
        ))}
```

```
    </div>
    {stats.invited.length >= 1 && !claimed && (
      <button onClick={claimAmbassador} style={{ marginTop:12, width:"100
        🌟 RÉCLAMER MON BADGE AMBASSADEUR
      </button>
    )}
    {claimed && (
      <div style={{ marginTop:12, background:D.goldSoft, border:`1px soli
        <div style={{ fontSize:13, fontFamily:D.heading, color:D.gold, le
      </div>
    )}
  </div>
</div>

{/* Partager */}
<div style={{ marginBottom:16 }}>
  <SLabel>PARTAGER TON INVITATION</SLabel>
  <div style={{ display:"flex", flexDirection:"column", gap:10 }}>
    <button onClick={shareInvite} style={{ width:"100%", border:"none", b
      <span style={{ fontSize:18 }}>📤</span> PARTAGER MON INVITATION
    </button>
    <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, bo
      <div style={{ fontSize:11, color:D.textMuted, marginBottom:8 }}>Env
      <div style={{ display:"flex", gap:8 }}>
        <input value={email} onChange={e=>setEmail(e.target.value)} place
        <button onClick={sendEmailInvite} style={{ background:sent?D.gree
          {sent ? "✅" : "ENVOYER"}
        </button>
      </div>
    </div>
  </div>
</div>

{/* Amis invités */}
{stats.invited.length > 0 && (
  <div>
    <SLabel>AMIS INVITÉS ({stats.invited.length})</SLabel>
    {stats.invited.map((e,i) => (
      <div key={i} style={{ background:D.bgCard, border:`1px solid ${D.bo
        <div style={{ width:32, height:32, borderRadius:"50%", background
        <div style={{ flex:1 }}>
          <div style={{ fontSize:12, color:D.textPrimary }}>{e}</div>
          <div style={{ fontSize:10, color:D.textMuted, marginTop:1 }}>7
        </div>
        <div style={{ fontSize:10, color:D.green, fontFamily:D.heading, l
      </div>
    ))}
```

```
          </div>
        )}

        {/* Message d'invitation */}
        <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
          <SLabel color={D.textMuted}>APERÇU DU MESSAGE</SLabel>
          <div style={{ fontSize:11, color:D.textSecondary, lineHeight:1.8, fontS
            {inviteText}
          </div>
        </div>
      </div>
    </div>
  );
}

// ── OFFLINE SYSTEM ──────────────────────────────────────────────
const OFFLINE_DEFIS = {
  "Débutant": [
    { titre:"Fondamentaux du Guerrier", emoji:"🌱", objectif:"Maîtrise les bases
    { titre:"Cardio Débutant", emoji:"💪", objectif:"Construis ton endurance pas
  ],
  "Intermédiaire": [
    { titre:"Combat Conditioning", emoji:"⚡", objectif:"Pousse tes limites et am
    { titre:"Technique Avancée", emoji:"🎯", objectif:"Perfectionne tes combinais
  ],
  "Avancé": [
    { titre:"Guerrier Elite", emoji:"🔥", objectif:"Dépasse tes limites — entraîn
    { titre:"Cardio Muay Thai", emoji:"💥", objectif:"Endurance maximale — va che
  ],
};

function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);
  useEffect(() => {
    const on = () => setIsOnline(true);
    const off = () => setIsOnline(false);
    window.addEventListener("online", on);
    window.addEventListener("offline", off);
    return () => { window.removeEventListener("online", on); window.removeEventLi
  }, []);
  return isOnline;
}

function OfflineBanner({ isOnline }) {
  if (isOnline) return null;
  return (
    <div style={{position: "absolute", top: 44, left: 0, right: 0, zIndex: 150, b
```

```
      <span style={{ fontSize: 14 }}>📵</span>
      <span style={{ fontSize: 12, color: "#000", fontFamily: "'Bebas Neue', curs
        MODE HORS LIGNE — Timer, techniques et progression disponibles
      </span>
    </div>
  );
}

function OfflineMessage({ feature }) {
  return (
    <div style={{background: "rgba(245,158,11,0.08)", border: "1px solid rgba(245
      <span style={{ fontSize: 24, flexShrink: 0 }}>📵</span>
      <div>
        <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursive", color: "
          MODE HORS LIGNE
        </div>
        <div style={{ fontSize: 12, color: "#888", lineHeight: 1.5 }}>
          {feature} nécessite une connexion internet. Reconnecte-toi pour utilise
        </div>
        <div style={{ fontSize: 11, color: "#666", marginTop: 6 }}>
          ✅ Timer · Techniques · Progression · Badges disponibles hors ligne
        </div>
      </div>
    </div>
  );
}

// ── SHARE SYSTEM ──────────────────────────────────────────────
function ShareCard({ title, subtitle, emoji, stats, color, onClose }) {
  const canvasRef = useRef(null);
  const [imageUrl, setImageUrl] = useState(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext("2d");
    canvas.width = 1080;
    canvas.height = 1080;

    const bg = ctx.createLinearGradient(0, 0, 1080, 1080);
    bg.addColorStop(0, "#0a0a0a");
    bg.addColorStop(0.5, "#1a0000");
    bg.addColorStop(1, "#0a0a0a");
    ctx.fillStyle = bg;
    ctx.fillRect(0, 0, 1080, 1080);

    ctx.beginPath();
```

```
ctx.arc(900, 180, 300, 0, Math.PI * 2);
ctx.strokeStyle = `${color}30`;
ctx.lineWidth = 2;
ctx.stroke();
ctx.beginPath();
ctx.arc(900, 180, 220, 0, Math.PI * 2);
ctx.strokeStyle = `${color}20`;
ctx.lineWidth = 1;
ctx.stroke();

const corners = [[60,60,1,1],[1020,60,-1,1],[60,1020,1,-1],[1020,1020,-1,-1]]
corners.forEach(([x,y,dx,dy]) => {
  ctx.beginPath();ctx.moveTo(x,y);ctx.lineTo(x+dx*50,y);
  ctx.strokeStyle = `${color}80`;ctx.lineWidth = 3;ctx.stroke();
  ctx.beginPath();ctx.moveTo(x,y);ctx.lineTo(x,y+dy*50);ctx.stroke();
});

ctx.font = "160px serif";
ctx.textAlign = "center";
ctx.fillText(emoji, 540, 380);

ctx.fillStyle = "#ffffff";
ctx.font = "bold 88px Arial, sans-serif";
ctx.letterSpacing = "8px";
ctx.fillText(title.toUpperCase(), 540, 500);

ctx.fillStyle = color;
ctx.font = "48px Arial, sans-serif";
ctx.fillText(subtitle, 540, 570);

if (stats && stats.length > 0) {
  const statW = 280;
  const startX = 540 - ((stats.length - 1) * statW / 2);
  stats.forEach((s, i) => {
    const x = startX + i * statW;
    ctx.fillStyle = `${color}20`;
    ctx.beginPath();
    ctx.roundRect(x - 110, 620, 220, 120, 16);
    ctx.fill();
    ctx.strokeStyle = `${color}50`;
    ctx.lineWidth = 1;
    ctx.stroke();
    ctx.fillStyle = "#ffffff";
    ctx.font = "bold 52px Arial";
    ctx.fillText(s.value, x, 690);
    ctx.fillStyle = "#888888";
    ctx.font = "28px Arial";
```

```
      ctx.fillText(s.label, x, 724);
    });
  }

  ctx.fillStyle = "#ef4444";
  ctx.font = "bold 52px Arial";
  ctx.fillText("AUTAM", 540, 860);
  ctx.fillStyle = "#ffffff";
  ctx.font = "36px Arial";
  ctx.fillText("MUAY THAI", 540, 910);

  ctx.fillStyle = "#444444";
  ctx.font = "italic 28px Georgia, serif";
  ctx.fillText("Forge ton corps · Affûte ton esprit", 540, 980);

  setImageUrl(canvas.toDataURL("image/png"));
}, []);

const shareImage = async () => {
  if (!imageUrl) return;
  try {
    const blob = await (await fetch(imageUrl)).blob();
    const file = new File([blob], "autam-muaythai.png", { type: "image/png" });
    if (navigator.share && navigator.canShare({ files: [file] })) {
      await navigator.share({ files: [file], title: "Autam Muay Thai", text: su
    } else {
      const a = document.createElement("a");
      a.href = imageUrl; a.download = "autam-muaythai.png"; a.click();
    }
  } catch {}
};

const shareText = async () => {
  const text = `🥊 ${title}\n${subtitle}\n\nEntraîne-toi avec Autam Muay Thai !
  try {
    if (navigator.share) {
      await navigator.share({ title: "Autam Muay Thai", text });
    } else {
      await navigator.clipboard.writeText(text);
      alert("Texte copié dans le presse-papiers !");
    }
  } catch {}
};

return (
  <div style={{ position: "absolute", inset: 0, zIndex: 200, background: "rgba(
    <canvas ref={canvasRef} style={{ display: "none" }} />
```

```
      <div style={{ display: "flex", justifyContent: "space-between", alignItems:
        <div style={{ fontSize: 18, fontFamily: "'Bebas Neue', cursive", color: "
        <button onClick={onClose} style={{ background: "rgba(255,255,255,0.06)",
      </div>

      {/* Preview */}
      {imageUrl && (
        <div style={{ marginBottom: 16, borderRadius: 16, overflow: "hidden", bor
          <img src={imageUrl} alt="share preview" style={{ width: "100%", display
        </div>
      )}
      {!imageUrl && (
        <div style={{ height: 200, background: "rgba(255,255,255,0.03)", borderRa
          <div style={{ fontSize: 13, color: "#555" }}>Génération en cours...</di
        </div>
      )}

      {/* Share buttons */}
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
        <button onClick={shareImage} style={{ width: "100%", border: "none", bord
          <span style={{ fontSize: 20 }}>📸</span> PARTAGER L'IMAGE (INSTAGRAM /
        </button>
        <button onClick={shareText} style={{ width: "100%", border: "1px solid rg
          <span style={{ fontSize: 20 }}>💬</span> PARTAGER VIA WHATSAPP / SMS
        </button>
      </div>

      <div style={{ textAlign: "center", marginTop: 12, fontSize: 10, color: "#44
        L'image sera sauvegardée dans ta galerie<br />avant d'être partagée
      </div>
    </div>
  );
}

function useShare() {
  const [shareData, setShareData] = useState(null);

  const shareProgress = (stats) => {
    const progStats = progression.getStats();
    setShareData({
      title: "Ma Progression",
      subtitle: `${progStats.sessions} séances · ${progStats.streak} jours de str
      emoji: "📈",
      color: "#22c55e",
      stats: [
        { value: String(progStats.sessions), label: "Séances" },
        { value: `${progStats.streak}j`, label: "Streak" },
```

```
        { value: `${progStats.totalCalories}`, label: "Calories" },
      ]
    });
  };

  const shareBadge = (badge) => {
    setShareData({
      title: badge.name,
      subtitle: `Badge débloqué sur Autam Muay Thai !`,
      emoji: badge.icon,
      color: "#f59e0b",
      stats: [{ value: badge.icon, label: badge.category }]
    });
  };

  const shareDefi = (defi) => {
    setShareData({
      title: "Défi Complété !",
      subtitle: defi.titre,
      emoji: defi.emoji || "⚡",
      color: "#ef4444",
      stats: [
        { value: `${defi.rounds}`, label: "Rounds" },
        { value: `~${defi.calories_estimees}`, label: "Calories" },
      ]
    });
  };

  const closeShare = () => setShareData(null);

  return { shareData, shareProgress, shareBadge, shareDefi, closeShare };
}

// ── COACH CHAT SCREEN ──────────────────────────────────────────
const QUICK_QUESTIONS = [
  "Comment améliorer ma garde ?",
  "Programme pour débutant ?",
  "Conseils nutrition avant entraînement ?",
  "Comment travailler le Teep ?",
  "Comment récupérer après une séance intense ?",
  "Quels exercices pour améliorer ma puissance ?",
];

function CoachChatScreen({ setScreen, isPremium, goPaywall, isOnline }) {
  const [messages, setMessages] = useState(() => {
    try { return JSON.parse(localStorage.getItem("amt_chat") || "[]"); } catch {
  });
```

```
const [input, setInput] = useState("");
const [loading, setLoading] = useState(false);
const messagesEndRef = useRef(null);
const inputRef = useRef(null);
const stats = progression.getStats();

useEffect(() => {
  messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
}, [messages]);

const saveMessages = (msgs) => {
  localStorage.setItem("amt_chat", JSON.stringify(msgs.slice(-50))); // Keep la
};

const sendMessage = async (text) => {
  if (!text.trim() || loading) return;
  if (!isPremium) { goPaywall(); return; }
  if (!isOnline) {
    const offlineMsg = { role: "assistant", content: "📵 Je suis hors ligne en
    setMessages(prev => [...prev, { role: "user", content: text, time: new Date
    setInput("");
    return;
  }

  const userMsg = { role: "user", content: text, time: new Date().toLocaleTimeS
  const newMessages = [...messages, userMsg];
  setMessages(newMessages);
  setInput("");
  setLoading(true);

  try {
    const history = newMessages.slice(-10).map(m => ({
      role: m.role === "user" ? "user" : "assistant",
      content: m.content,
    }));

    const res = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514", max_tokens: 600,
        system: `Tu es un coach expert en Muay Thai, nutrition sportive et prép
        messages: history,
      })
    });
    const data = await res.json();
    const reply = data.content?.map(c => c.text || "").join("") || "Désolé, je
    const aiMsg = { role: "assistant", content: reply, time: new Date().toLocal
```

```
    const finalMessages = [...newMessages, aiMsg];
    setMessages(finalMessages);
    saveMessages(finalMessages);
  } catch {
    const errMsg = { role: "assistant", content: "Connexion impossible. Vérifie
    const finalMessages = [...newMessages, errMsg];
    setMessages(finalMessages);
    saveMessages(finalMessages);
  }
  setLoading(false);
};

const clearChat = () => {
  setMessages([]);
  localStorage.removeItem("amt_chat");
};

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    {/* Header */}
    <div style={{ background: "#0a0a0a", padding: "16px", borderBottom: "1px so
      <button onClick={() => setScreen("home")} style={{ background: "rgba(255,
      <div style={{ width: 42, height: 42, borderRadius: "50%", background: "li
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 16, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ display: "flex", alignItems: "center", gap: 5, marginTop:
          <div style={{ width: 6, height: 6, borderRadius: "50%", background: "
          <span style={{ fontSize: 10, color: "#22c55e" }}>En ligne · Coach IA
        </div>
      </div>
      {messages.length > 0 && (
        <button onClick={clearChat} style={{ background: "none", border: "none"
      )}
    </div>

    {/* Premium lock */}
    {!isPremium && (
      <div style={{ flex: 1, display: "flex", flexDirection: "column", alignIte
        <div style={{ fontSize: 60, marginBottom: 16 }}>🔒</div>
        <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 13, color: "#888", marginBottom: 24, lineHeight
          Pose toutes tes questions à ton coach IA personnel — techniques, nutr
        </div>
        <button onClick={goPaywall} style={{ width: "100%", border: "none", bor
          ⭐ DÉBLOQUER LE CHAT
        </button>
      </div>
```

```
)}

{/* Messages */}
{isPremium && (
  <>
    <div style={{ flex: 1, overflowY: "auto", padding: "12px 16px" }}>
      {/* Message de bienvenue */}
      {messages.length === 0 && (
        <div style={{ marginBottom: 20 }}>
          <div style={{ display: "flex", gap: 10, marginBottom: 16 }}>
            <div style={{ width: 34, height: 34, borderRadius: "50%", backg
            <div style={{ background: "rgba(255,255,255,0.06)", borderRadiu
              <div style={{ fontSize: 13, color: "#fff", lineHeight: 1.6, m
                Salut ! Je suis ton Coach Autam 🥊 Je suis là pour t'aider
              </div>
              <div style={{ fontSize: 13, color: "#ddd", lineHeight: 1.6 }}
                Pose-moi n'importe quelle question, je suis là ! 💪
              </div>
              <div style={{ fontSize: 10, color: "#555", marginTop: 6 }}>Co
            </div>
          </div>

          {/* Questions rapides */}
          <div style={{ fontSize: 11, color: "#555", letterSpacing: 2, font
          <div style={{ display: "flex", flexDirection: "column", gap: 6 }}
            {QUICK_QUESTIONS.map(q => (
              <button key={q} onClick={() => sendMessage(q)} style={{ backg
                💬 {q}
              </button>
            ))}
          </div>
        </div>
      )}

      {/* Messages */}
      {messages.map((msg, i) => (
        <div key={i} style={{ display: "flex", gap: 10, marginBottom: 14, f
          {msg.role === "assistant" && (
            <div style={{ width: 30, height: 30, borderRadius: "50%", backg
          )}
          <div style={{ maxWidth: "78%" }}>
            <div style={{background: msg.role === "user" ? "#ef4444" : "rgb
              <div style={{ fontSize: 13, color: "#fff", lineHeight: 1.65 }
            </div>
            <div style={{ fontSize: 10, color: "#444", marginTop: 4, textAl
          </div>
        </div>
```

```
            ))}

            {/* Loading dots */}
            {loading && (
              <div style={{ display: "flex", gap: 10, marginBottom: 14 }}>
                <div style={{ width: 30, height: 30, borderRadius: "50%", backgro
                <div style={{ background: "rgba(255,255,255,0.06)", borderRadius:
                  <style>{`@keyframes dot{0%,80%,100%{opacity:0.3}40%{opacity:1}}
                  <div style={{ display: "flex", gap: 4 }}>
                    {[0,1,2].map(i => <div key={i} style={{ width: 6, height: 6,
                  </div>
                </div>
              </div>
            )}
            <div ref={messagesEndRef}/>
          </div>

          {/* Input */}
          <div style={{ padding: "10px 12px 20px", borderTop: "1px solid rgba(255
            <textarea
              ref={inputRef}
              value={input}
              onChange={e => setInput(e.target.value)}
              onKeyDown={e => { if(e.key === "Enter" && !e.shiftKey) { e.preventD
              placeholder="Pose ta question au Coach Autam..."
              style={{ flex: 1, background: "rgba(255,255,255,0.06)", border: "1p
              rows={1}
            />
            <button onClick={() => sendMessage(input)} disabled={!input.trim() ||
              ↑
            </button>
          </div>
        </>
      )}
    </div>
  );
}

// ── DÉFI DU JOUR ──────────────────────────────────────────────
const defiStorage = {
  get: () => { try { return JSON.parse(localStorage.getItem("amt_defi") || "null"
  set: (d) => localStorage.setItem("amt_defi", JSON.stringify(d)),
  isToday: (d) => d && new Date(d.date).toDateString() === new Date().toDateStrin
};

function DefiDuJourScreen({ setScreen, isPremium, goPaywall, onBadge, onShare, is
  const [level, setLevel] = useState(() => localStorage.getItem("amt_defi_level")
```

```
  const [defi, setDefi] = useState(() => {
    const saved = defiStorage.get();
    return defiStorage.isToday(saved) ? saved : null;
  });
  const [loading, setLoading] = useState(false);
  const [completed, setCompleted] = useState(false);
  const [showTimer, setShowTimer] = useState(false);
  const [timerRound, setTimerRound] = useState(1);
  const [timeLeft, setTimeLeft] = useState(0);
  const [timerRunning, setTimerRunning] = useState(false);
  const intRef = useRef(null);
  const stats = progression.getStats();

  const levelColors = { "Débutant": "#22c55e", "Intermédiaire": "#f59e0b", "Avanc
  const color = levelColors[level] || "#ef4444";

  const generateDefi = async () => {
    if (!isPremium) { goPaywall(); return; }
    setLoading(true);
    if (!isOnline) {
      const offlineList = OFFLINE_DEFIS[level] || OFFLINE_DEFIS["Débutant"];
      const fallback = { ...offlineList[Math.floor(Math.random() * offlineList.le
      defiStorage.set(fallback);
      setDefi(fallback);
      setLoading(false);
      return;
    }
    try {
      const statsInfo = `${stats.sessions} séances, ${stats.streak} jours de stre
      const res = await fetch("https://api.anthropic.com/v1/messages", {
        method: "POST", headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          model: "claude-sonnet-4-20250514", max_tokens: 800,
          system: `Tu es un coach Muay Thai expert qui crée des défis d'entraînem
{"titre":"<nom accrocheur du défi>","emoji":"<1 emoji>","objectif":"<une phrase m
          messages: [{
            role: "user",
            content: `Crée un défi du jour en Muay Thai pour niveau ${level}.
Progression de l'élève: ${statsInfo}.
Date: ${new Date().toLocaleDateString("fr-FR")}.
Adapte la difficulté au niveau ${level}.
Le combo doit avoir 3-5 frappes.
Les exercices doivent être 3-4 exercices complémentaires au Muay Thai.
JSON uniquement, pas de markdown.`
          }]
        })
      });
```

```
    const data = await res.json();
    const text = data.content?.map(c => c.text || "").join("") || "";
    const parsed = JSON.parse(text.replace(/```json|```/g, "").trim());
    const newDefi = { ...parsed, date: new Date().toISOString(), level };
    defiStorage.set(newDefi);
    setDefi(newDefi);
  } catch {
    const fallback = {
      titre: "Combat du Guerrier", emoji: "🥊",
      objectif: "Forge ta technique et ta résistance aujourd'hui !",
      niveau: level, date: new Date().toISOString(), level,
      combo: ["Jab", "Cross", "Crochet", "Teep"],
      rounds: level === "Débutant" ? 3 : level === "Intermédiaire" ? 5 : 8,
      duree_round: level === "Débutant" ? 90 : level === "Intermédiaire" ? 120
      repos: level === "Débutant" ? 60 : 45,
      exercices: [
        { nom: "Shadowboxing", reps: "3 min", desc: "Travaille ta garde et tes
        { nom: "Genoux sur sac", reps: "4x20", desc: "Monte le genou haut, rota
        { nom: "Gainage", reps: "3x45s", desc: "Core solide = frappes puissante
      ],
      conseil_coach: "Concentre-toi sur la qualité plutôt que la quantité. Chaq
      calories_estimees: level === "Débutant" ? 250 : level === "Intermédiaire"
    };
    defiStorage.set(fallback);
    setDefi(fallback);
  }
  setLoading(false);
};

const startDefiTimer = () => {
  setShowTimer(true);
  setTimerRound(1);
  setTimeLeft(defi.duree_round);
};

useEffect(() => {
  if (timerRunning && showTimer) {
    intRef.current = setInterval(() => {
      setTimeLeft(t => {
        if (t <= 1) {
          if (timerRound >= defi?.rounds) {
            clearInterval(intRef.current);
            setTimerRunning(false);
            setShowTimer(false);
            setCompleted(true);
            const { newBadges } = progression.addSession(Math.round((defi.round
            if (newBadges.length > 0) setTimeout(() => onBadge(newBadges[0]), 5
```

```
            return 0;
          }
          setTimerRound(r => r + 1);
          return defi.repos;
        }
        return t - 1;
      });
    }, 1000);
  }
  return () => clearInterval(intRef.current);
}, [timerRunning, showTimer, timerRound]);

const fmt = s => `${String(Math.floor(s / 60)).padStart(2, "0")}:${String(s % 6
const isRest = timerRound > 1 && timeLeft > defi?.duree_round - 5;

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    <style>{`@keyframes celebrate{0%{transform:scale(1)}50%{transform:scale(1.0

    {/* Header */}
    <div style={{ background: "#0a0a0a", padding: "20px 16px 16px", borderBotto
      <button onClick={() => setScreen("home")} style={{ background: "rgba(255,
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 20, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 10, color: "#666" }}>{new Date().toLocaleDateSt
      </div>
      {defi && <div style={{ fontSize: 10, color: color, fontFamily: "'Bebas Ne
    </div>

    {/* Timer overlay */}
    {showTimer && defi && (
      <div style={{ position: "absolute", inset: 0, zIndex: 100, background: "#
        <div style={{ fontSize: 13, color: color, fontFamily: "'Bebas Neue', cu
          {timerRound <= defi.rounds ? `ROUND ${timerRound} / ${defi.rounds}` :
        </div>
        <div style={{ fontSize: 80, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 13, color: "#666", marginBottom: 32, fontStyle:
        <div style={{ display: "flex", gap: 16 }}>
          <button onClick={() => { clearInterval(intRef.current); setTimerRunni
            {timerRunning ? "⏸" : "▶"}
          </button>
          <button onClick={() => { clearInterval(intRef.current); setShowTimer(
        </div>
      </div>
    )}

    <div style={{ flex: 1, overflowY: "auto", padding: 16 }}>
```

```
{/* Completed */}
{completed && (
  <div style={{ background: `${color}15`, border: `1px solid ${color}40`,
    <div style={{ fontSize: 48, marginBottom: 8 }}>🏆</div>
    <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", colo
    <div style={{ fontSize: 13, color: "#888" }}>~{defi?.calories_estimee
  </div>
)}

{/* Sélecteur de niveau */}
{!defi && (
  <div style={{ marginBottom: 20 }}>
    <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontF
    <div style={{ display: "flex", gap: 8 }}>
      {["Débutant", "Intermédiaire", "Avancé"].map(l => (
        <button key={l} onClick={() => { setLevel(l); localStorage.setIte
          {l === "Débutant" ? "🌱" : l === "Intermédiaire" ? "⚡" : "🔥"
        </button>
      ))}
    </div>
  </div>
)}

{/* Pas de défi — bouton générer */}
{!defi && (
  <div style={{ textAlign: "center", padding: "32px 0" }}>
    <div style={{ fontSize: 64, marginBottom: 16 }}>⚡</div>
    <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", colo
    <div style={{ fontSize: 13, color: "#666", marginBottom: 28, lineHeig
      L'IA génère un défi unique<br />adapté à ton niveau et ta progressi
    </div>
    {!isPremium && (
      <div style={{ background: "rgba(245,158,11,0.08)", border: "1px sol
        <div style={{ fontSize: 12, color: "#f59e0b", fontFamily: "'Bebas
        <div style={{ fontSize: 11, color: "#666" }}>Le défi IA est réser
      </div>
    )}
    <button onClick={generateDefi} disabled={loading} style={{width: "100
      {loading ? "⏳ L'IA PRÉPARE TON DÉFI..." : "⚡ GÉNÉRER MON DÉFI"}
    </button>
  </div>
)}

{/* Défi affiché */}
{defi && !loading && (
  <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
```

```
{/* Hero du défi */}
<div style={{ background: `linear-gradient(135deg, ${color}20, ${colo
  <div style={{ fontSize: 44, marginBottom: 8 }}>{defi.emoji}</div>
  <div style={{ fontSize: 26, fontFamily: "'Bebas Neue', cursive", co
  <div style={{ fontSize: 13, color: "#aaa", fontStyle: "italic", lin
  <div style={{ display: "flex", justifyContent: "center", gap: 16, m
    {[
      { icon: "🔄", val: `${defi.rounds} rounds` },
      { icon: "⏱️", val: `${defi.duree_round}s` },
      { icon: "💪", val: `~${defi.calories_estimees} kcal` },
    ].map(s => (
      <div key={s.val} style={{ textAlign: "center" }}>
        <div style={{ fontSize: 16 }}>{s.icon}</div>
        <div style={{ fontSize: 11, color: "#888", marginTop: 2 }}>{s
      </div>
    ))}
  </div>
</div>

{/* Combo */}
<div style={{ background: "rgba(255,255,255,0.02)", border: "1px soli
  <div style={{ fontSize: 11, color: color, letterSpacing: 3, fontFam
  <div style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
    {defi.combo.map((f, i) => (
      <div key={i} style={{ display: "flex", alignItems: "center", ga
        <span style={{ background: `${color}20`, border: `1px solid $
        {i < defi.combo.length - 1 && <span style={{ color: "#444" }}
      </div>
    ))}
  </div>
</div>

{/* Exercices */}
<div style={{ background: "rgba(255,255,255,0.02)", border: "1px soli
  <div style={{ fontSize: 11, color: color, letterSpacing: 3, fontFam
  {defi.exercices?.map((ex, i) => (
    <div key={i} style={{ marginBottom: 12, paddingBottom: 10, border
      <div style={{ display: "flex", justifyContent: "space-between",
        <div style={{ fontSize: 14, fontFamily: "'Bebas Neue', cursiv
        <span style={{ fontSize: 12, color, fontFamily: "'Bebas Neue'
      </div>
      <div style={{ fontSize: 11, color: "#777", fontStyle: "italic"
    </div>
  ))}
</div>

{/* Conseil coach */}
```

```
            <div style={{ background: "rgba(168,85,247,0.06)", border: "1px solid
              <div style={{ fontSize: 11, color: "#a855f7", letterSpacing: 3, fon
              <div style={{ fontSize: 12, color: "#ccc", lineHeight: 1.7, fontSty
            </div>

            {/* Boutons action */}
            {!completed && (
              <button onClick={startDefiTimer} style={{width: "100%", border: "no
            )}

          {completed && (
            <button onClick={() => onShare && onShare(defi)} style={{ width: "100
              📩 PARTAGER MON DÉFI
            </button>
          )}
            <button onClick={() => { setDefi(null); defiStorage.set(null); }} sty
          </div>
        )}
      </div>
    </div>
  );
}

// ── BADGES SYSTEM ──────────────────────────────────────────────


function BadgeNotification({ badge, onClose }) {
  useEffect(() => { const t = setTimeout(onClose, 4000); return () => clearTimeou
  return (
    <div style={{position: "absolute", top: 56, left: 16, right: 16, zIndex: 200,
      <style>{`@keyframes badgeSlide{from{opacity:0;transform:translateY(-20px)}t
      <div style={{width: 52, height: 52, borderRadius: "50%", flexShrink: 0, bac
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 10, color: "#f59e0b", fontFamily: "'Bebas Neue',
          🏆 BADGE DÉBLOQUÉ !
        </div>
        <div style={{ fontSize: 15, fontFamily: "'Bebas Neue', cursive", color: "
        <div style={{ fontSize: 11, color: "#888", marginTop: 2 }}>{badge.desc}</
      </div>
      <button onClick={onClose} style={{ background: "none", border: "none", colo
    </div>
  );
}

function BadgesScreen({ setScreen, onShare }) {
  const unlocked = badgeStorage.getUnlocked();
  const stats = badgeStorage.getStats();
```

```
const categories = [...new Set(ALL_BADGES.map(b => b.category))];

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    <div style={{ background: "#0a0a0a", padding: "20px 16px 16px", borderBotto
      <button onClick={() => setScreen("profile")} style={{ background: "rgba(2
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 20, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 10, color: "#666" }}>{unlocked.length} / {ALL_B
      </div>
      {unlocked.length > 0 && (
        <button onClick={() => onShare && onShare(ALL_BADGES.find(b => b.id ===
      )}
    </div>

    <div style={{ flex: 1, overflowY: "auto", padding: 16 }}>
      {/* Progress bar */}
      <div style={{ background: "rgba(255,255,255,0.03)", border: "1px solid rg
        <div style={{ display: "flex", justifyContent: "space-between", marginB
          <span style={{ fontSize: 12, color: "#aaa" }}>Progression globale</sp
          <span style={{ fontSize: 12, color: "#f59e0b", fontFamily: "'Bebas Ne
        </div>
        <div style={{ height: 6, background: "rgba(255,255,255,0.06)", borderRa
          <div style={{ height: "100%", width: `${unlocked.length/ALL_BADGES.le
        </div>
        <div style={{ display: "flex", gap: 16, marginTop: 12 }}>
          {[
            { label: "Séances", value: stats.sessions, icon: "🥊" },
            { label: "Streak", value: `${stats.streak}j`, icon: "🔥" },
            { label: "Analyses", value: stats.analyses, icon: "🤖" },
          ].map(s => (
            <div key={s.label} style={{ display: "flex", alignItems: "center",
              <span style={{ fontSize: 12 }}>{s.icon}</span>
              <span style={{ fontSize: 11, color: "#666" }}>{s.value} {s.label}
            </div>
          ))}
        </div>
      </div>

      {/* Badges par catégorie */}
      {categories.map(cat => (
        <div key={cat} style={{ marginBottom: 24 }}>
          <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontF
          <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr 1fr", ga
            {ALL_BADGES.filter(b => b.category === cat).map(b => {
              const isUnlocked = unlocked.includes(b.id);
              return (
```

```
                  <div key={b.id} style={{background: isUnlocked ? "rgba(245,158,
                    <div style={{fontSize: 30, marginBottom: 6, filter: isUnlocke
                    <div style={{ fontSize: 10, fontFamily: "'Bebas Neue', cursiv
                      {b.name}
                    </div>
                    <div style={{ fontSize: 9, color: isUnlocked ? "#888" : "#333
                      {b.desc}
                    </div>
                    {isUnlocked && (
                      <div style={{ marginTop: 6, fontSize: 8, color: "#f59e0b",
                    )}
                  </div>
                );
              })}
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}

// ── PROGRESSION SYSTEM ──────────────────────────────────────────
// ── SESSION COMPLETE MODAL ──────────────────────────────────────
function SessionCompleteModal({ minutes, onClose, onBadge }) {
  const [saved, setSaved] = useState(false);
  const cal = Math.round(minutes * 8.5);

  const saveSession = () => {
    const { newBadges } = progression.addSession(minutes);
    setSaved(true);
    if(newBadges.length > 0) setTimeout(() => onBadge(newBadges[0]), 500);
    setTimeout(onClose, 1800);
  };

  return (
    <div style={{position: "absolute", inset: 0, zIndex: 100, background: "rgba(1
      <style>{`@keyframes popIn{from{transform:scale(0.7);opacity:0}to{transform:
      <div style={{ animation: "popIn 0.4s ease", width: "100%" }}>
        <div style={{ fontSize: 60, marginBottom: 12 }}>{saved ? "🏆" : "🎉"}</di
        <div style={{ fontSize: 28, fontFamily: "'Bebas Neue', cursive", color: "
          {saved ? "SAUVEGARDÉ !" : "SÉANCE TERMINÉE !"}
        </div>
        {!saved && (
          <>
            <div style={{ fontSize: 13, color: "#888", marginBottom: 24, fontStyl
              Excellent travail ! Voici ton résumé
```

```
            </div>
            <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 1
              {[
                { icon: "⏱️", value: `${minutes} min`, label: "Durée" },
                { icon: "🔥", value: `~${cal} kcal`, label: "Calories" },
              ].map(s => (
                <div key={s.label} style={{ background: "rgba(255,255,255,0.04)",
                  <div style={{ fontSize: 28, marginBottom: 6 }}>{s.icon}</div>
                  <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive"
                  <div style={{ fontSize: 11, color: "#666", marginTop: 2 }}>{s.l
                </div>
              ))}
            </div>
            <button onClick={saveSession} style={{width: "100%", border: "none",
            <button onClick={onClose} style={{width: "100%", border: "1px solid r
          </>
        )}
      </div>
    </div>
  );
}

// ── PROGRESSION SCREEN ──────────────────────────────────────────
// ── STATS SCREEN (avancé) ──────────────────────────────────────
function StatsScreen({ setScreen }) {
  const stats = progression.getStats();
  const history = stats.history || [];
  const [period, setPeriod] = useState("30"); // 7 | 30 | 90

  // Records personnels
  const calculateRecords = () => {
    if (!history.length) return { longestStreak:0, bestMonth:0, bestDay:0, avgPer

    // Plus long streak (calcul depuis l'historique)
    const dates = [...new Set(history.map(h => h.date))].sort();
    let longestStreak = 0, currentStreak = 0, lastDate = null;
    dates.forEach(dateStr => {
      const [d,m,y] = dateStr.split("/").map(Number);
      const date = new Date(y, m-1, d);
      if (!lastDate) { currentStreak = 1; }
      else {
        const diff = Math.round((date - lastDate) / 864e5);
        if (diff === 1) currentStreak++;
        else currentStreak = 1;
      }
      longestStreak = Math.max(longestStreak, currentStreak);
      lastDate = date;
```

```
  });

  // Meilleur mois
  const monthCounts = {};
  history.forEach(h => {
    const [d,m,y] = h.date.split("/");
    const key = `${m}/${y}`;
    monthCounts[key] = (monthCounts[key] || 0) + 1;
  });
  const bestMonth = Math.max(0, ...Object.values(monthCounts));

  // Meilleur jour (max sessions en un jour)
  const dayCounts = {};
  history.forEach(h => { dayCounts[h.date] = (dayCounts[h.date] || 0) + 1; });
  const bestDay = Math.max(0, ...Object.values(dayCounts));

  // Moyenne par semaine
  const totalDays = dates.length;
  const weeks = Math.max(1, totalDays / 7);
  const avgPerWeek = Math.round((stats.sessions / weeks) * 10) / 10;

  // Type le plus fréquent
  const typeCounts = {};
  history.forEach(h => { typeCounts[h.type] = (typeCounts[h.type] || 0) + 1; })
  const mostType = Object.entries(typeCounts).sort((a,b) => b[1]-a[1])[0]?.[0]

  return { longestStreak, bestMonth, bestDay, avgPerWeek, totalDays, mostType }
};

const records = calculateRecords();

// Évolution sur N jours (graphique ligne)
const evolution = (() => {
  const days = parseInt(period);
  const data = [];
  const today = new Date();
  today.setHours(0,0,0,0);
  for (let i = days - 1; i >= 0; i--) {
    const d = new Date(today);
    d.setDate(d.getDate() - i);
    const ds = d.toLocaleDateString("fr-FR");
    const count = history.filter(h => h.date === ds).length;
    data.push({ date:d, count, ds });
  }
  return data;
})();
```

```
const maxEvol = Math.max(1, ...evolution.map(e => e.count));

// Heatmap GitHub style — 90 derniers jours
const heatmap = (() => {
  const data = [];
  const today = new Date();
  today.setHours(0,0,0,0);
  for (let i = 89; i >= 0; i--) {
    const d = new Date(today);
    d.setDate(d.getDate() - i);
    const ds = d.toLocaleDateString("fr-FR");
    const count = history.filter(h => h.date === ds).length;
    data.push({ date:d, count });
  }
  return data;
})();

// Répartition par type
const typeStats = (() => {
  const counts = {};
  history.forEach(h => { counts[h.type] = (counts[h.type] || 0) + 1; });
  const total = Object.values(counts).reduce((s,n) => s+n, 0);
  return Object.entries(counts).sort((a,b) => b[1]-a[1]).map(([type, count]) =>
    type, count, pct: total ? Math.round((count/total)*100) : 0,
  }));
})();

const typeColors = ["#ef4444", "#f59e0b", "#22c55e", "#a855f7", "#06b6d4", "#ec

return (
  <div style={{ height:"100%", display:"flex", flexDirection:"column", backgrou
    {/* Header */}
    <div style={{ background:D.bg, padding:"20px 16px 16px", borderBottom:`1px
      <button onClick={() => setScreen("progression")} style={{ background:D.bg
      <div>
        <div style={{ fontSize:20, fontFamily:D.heading, color:D.textPrimary, l
        <div style={{ fontSize:10, color:D.textMuted }}>Records · graphiques ·
      </div>
    </div>

    <div style={{ flex:1, overflowY:"auto", padding:16 }}>

      {/* Records personnels */}
      <SLabel>🎖️ RECORDS PERSONNELS</SLabel>
      <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:8, margi
        {[
          { icon:"🔥", label:"Plus long streak", value:`${records.longestStreak
```

```
    { icon:"📅", label:"Meilleur mois", value:`${records.bestMonth}`, col
    { icon:"⚡", label:"Meilleur jour", value:`${records.bestDay}`, color
    { icon:"📈", label:"Moy/semaine", value:`${records.avgPerWeek}`, colc
  ].map(r => (
    <div key={r.label} style={{ background:`${r.color}10`, border:`1px so
      <div style={{ fontSize:22, marginBottom:6 }}>{r.icon}</div>
      <div style={{ fontSize:24, fontFamily:D.heading, color:r.color, lin
      <div style={{ fontSize:10, color:D.textMuted, marginTop:4 }}>{r.sub
    </div>
  ))}
</div>

{/* Évolution */}
<div style={{ display:"flex", justifyContent:"space-between", alignItems:
  <SLabel style={{ marginBottom:0 }}>📈 ÉVOLUTION</SLabel>
  <div style={{ display:"flex", gap:6 }}>
    {[{v:"7",l:"7j"},{v:"30",l:"30j"},{v:"90",l:"90j"}].map(p => (
      <button key={p.v} onClick={() => setPeriod(p.v)} style={{
        background: period===p.v ? D.red : "transparent",
        border: `1px solid ${period===p.v ? D.red : D.border}`,
        color: period===p.v ? "#fff" : D.textMuted,
        borderRadius:20, padding:"4px 10px", cursor:"pointer",
        fontSize:10, fontFamily:D.heading, letterSpacing:1,
      }}>{p.l}</button>
    ))}
  </div>
</div>
<div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
  {history.length === 0 ? (
    <div style={{ textAlign:"center", padding:"20px 0", color:D.textMuted
      Aucune séance enregistrée
    </div>
  ) : (
    <>
      <div style={{ display:"flex", alignItems:"flex-end", gap:2, height:
        {evolution.map((e, i) => (
          <div key={i} style={{ flex:1, display:"flex", flexDirection:"co
            <div style={{
              width:"100%",
              height:`${(e.count/maxEvol)*72}px`,
              minHeight: e.count > 0 ? 3 : 0,
              background: e.count > 0 ? `linear-gradient(180deg,${D.red},
              borderRadius:2,
              transition:"all 0.3s",
            }}/>
          </div>
        ))}
```

```
      </div>
      <div style={{ display:"flex", justifyContent:"space-between", fontS
        <span>{evolution[0]?.date.toLocaleDateString("fr-FR",{day:"numeri
        <span>Aujourd'hui</span>
      </div>
    </>
  )}
</div>

{/* Heatmap GitHub style */}
<SLabel>🔥 ACTIVITÉ DES 90 DERNIERS JOURS</SLabel>
<div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
  <div style={{ display:"grid", gridTemplateColumns:"repeat(15, 1fr)", ga
    {heatmap.map((d, i) => {
      const intensity = Math.min(d.count, 4);
      const colors = ["rgba(255,255,255,0.04)", `${D.red}40`, `${D.red}70
      return (
        <div key={i} title={`${d.date.toLocaleDateString("fr-FR")} : ${d.
          aspectRatio:"1", borderRadius:3,
          background: colors[intensity],
          border:`1px solid ${intensity > 0 ? D.borderHi : D.border}`,
          boxShadow: intensity >= 3 ? `0 0 4px ${D.red}80` : "none",
        }}/>
      );
    })}
  </div>
  <div style={{ display:"flex", justifyContent:"space-between", alignItem
    <span>Moins</span>
    <div style={{ display:"flex", gap:3 }}>
      {[0,1,2,3,4].map(i => (
        <div key={i} style={{ width:12, height:12, borderRadius:3, backgr
      ))}
    </div>
    <span>Plus</span>
  </div>
</div>

{/* Répartition par type */}
<SLabel>🍩 RÉPARTITION PAR TYPE</SLabel>
<div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
  {typeStats.length === 0 ? (
    <div style={{ textAlign:"center", padding:"20px 0", color:D.textMuted
      Pas encore assez de données
    </div>
  ) : (
    typeStats.map((t, i) => (
      <div key={t.type} style={{ marginBottom:i<typeStats.length-1?12:0 }
```

```
                <div style={{ display:"flex", justifyContent:"space-between", mar
                  <span style={{ fontSize:12, color:D.textPrimary, fontFamily:D.h
                  <span style={{ fontSize:11, color:typeColors[i%6], fontFamily:D
                </div>
                <div style={{ height:6, background:"rgba(255,255,255,0.06)", bord
                  <div style={{ height:"100%", width:`${t.pct}%`, background:type
                </div>
              </div>
            ))
          )}
        </div>

        {/* Stats globales */}
        <SLabel>📊 STATISTIQUES GLOBALES</SLabel>
        <div style={{ background:D.bgCard, border:`1px solid ${D.border}`, border
          {[
            { label:"Total séances", value:stats.sessions || 0 },
            { label:"Total minutes", value:`${stats.totalMinutes || 0}min` },
            { label:"Total calories", value:`${stats.totalCalories || 0}` },
            { label:"Jours actifs", value:records.totalDays },
            { label:"Type favori", value:records.mostType },
          ].map((s, i, arr) => (
            <div key={s.label} style={{ display:"flex", justifyContent:"space-bet
              <span style={{ fontSize:12, color:D.textMuted }}>{s.label}</span>
              <span style={{ fontSize:13, color:D.textPrimary, fontFamily:D.headi
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}


function ProgressionScreen({ setScreen, onBadge, onShare }) {
  const [stats, setStats] = useState(progression.getStats());
  const [showManual, setShowManual] = useState(false);
  const [manualMinutes, setManualMinutes] = useState(30);
  const [manualType, setManualType] = useState("Entraînement général");
  const [justSaved, setJustSaved] = useState(false);

  const hours = Math.floor(stats.totalMinutes / 60);
  const mins = stats.totalMinutes % 60;
  const weekly = stats.weeklySessionss || [0,0,0,0,0,0,0];
  const maxW = Math.max(...weekly, 1);
  const today = new Date().getDay();
```

```
const addManual = () => {
  const { newBadges } = progression.addSession(manualMinutes, manualType);
  setStats(progression.getStats());
  setShowManual(false);
  setJustSaved(true);
  setTimeout(() => setJustSaved(false), 2000);
  if(newBadges.length > 0) setTimeout(() => onBadge(newBadges[0]), 500);
};

const types = ["Entraînement général", "Cardio", "Technique", "Sparring", "Sac

return (
  <div style={{ height: "100%", display: "flex", flexDirection: "column", backg
    {/* Header */}
    <div style={{ background: "#0a0a0a", padding: "20px 16px 16px", borderBotto
      <button onClick={() => setScreen("profile")} style={{ background: "rgba(2
      <div style={{ flex: 1 }}>
        <div style={{ fontSize: 20, fontFamily: "'Bebas Neue', cursive", color:
        <div style={{ fontSize: 10, color: "#666" }}>Suivi de tes entraînements
      </div>
      {justSaved && <div style={{ fontSize: 11, color: "#22c55e", fontFamily: "
      <button onClick={() => setScreen("stats")} style={{ background: "rgba(168
      <button onClick={() => onShare && onShare()} style={{ background: "rgba(3
    </div>

    <div style={{ flex: 1, overflowY: "auto", padding: 16 }}>
      {/* Stats principales */}
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10, m
        {[
          { icon: "🥊", value: stats.sessions, label: "Séances", color: "#ef444
          { icon: "🔥", value: `${stats.streak}j`, label: "Streak actuel", colc
          { icon: "⏱️", value: hours > 0 ? `${hours}h${mins>0?mins+"m":""}` : `
          { icon: "💪", value: `${stats.totalCalories}`, label: "Calories brûlé
        ].map(s => (
          <div key={s.label} style={{ background: `${s.color}10`, border: `1px
            <div style={{ fontSize: 24, marginBottom: 6 }}>{s.icon}</div>
            <div style={{ fontSize: 22, fontFamily: "'Bebas Neue', cursive", co
            <div style={{ fontSize: 10, color: "#666", marginTop: 2 }}>{s.label
          </div>
        ))}
      </div>

      {/* Bouton Stats avancées */}
      <button onClick={() => setScreen("stats")} style={{ width:"100%", border:
        <span style={{ fontSize:16 }}>📊</span> VOIR LES STATS AVANCÉES
      </button>
```

```
{/* Activité semaine */}
<div style={{ background: "rgba(255,255,255,0.02)", border: "1px solid rg
  <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFam
  <div style={{ display: "flex", alignItems: "flex-end", gap: 6, height:
    {["D","L","M","M","J","V","S"].map((d,i) => {
      const val = weekly[i] || 0;
      const h = Math.max(4, (val/maxW)*52);
      const isToday = i === today;
      return (
        <div key={i} style={{ flex: 1, display: "flex", flexDirection: "c
          <div style={{ width: "100%", borderRadius: 4, height: `${h}px`,
          <span style={{ fontSize: 9, color: isToday ? "#ef4444" : "#555"
        </div>
      );
    })}
  </div>
</div>

{/* Ajouter séance manuelle */}
<div style={{ background: "rgba(239,68,68,0.06)", border: "1px solid rgba
  <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontFam
  {!showManual ? (
    <button onClick={() => setShowManual(true)} style={{width: "100%", bo
  ) : (
    <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
      <div>
        <div style={{ fontSize: 11, color: "#888", marginBottom: 6 }}>Typ
        <div style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
          {types.map(t => (
            <button key={t} onClick={() => setManualType(t)} style={{back
          ))}
        </div>
      </div>
      <div>
        <div style={{ display: "flex", justifyContent: "space-between", m
          <span style={{ fontSize: 11, color: "#888" }}>Durée</span>
          <span style={{ fontSize: 12, color: "#ef4444", fontFamily: "'Be
        </div>
        <input type="range" min={5} max={120} step={5} value={manualMinut
      </div>
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap:
        <button onClick={() => setShowManual(false)} style={{ background:
        <button onClick={addManual} style={{ background: "#ef4444", borde
      </div>
    </div>
  )}
</div>
```

```
        {/* Historique */}
        {stats.history && stats.history.length > 0 && (
          <div>
            <div style={{ fontSize: 11, color: "#ef4444", letterSpacing: 3, fontF
            <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
              {stats.history.slice(0, 10).map(h => (
                <div key={h.id} style={{ background: "rgba(255,255,255,0.02)", bo
                  <div style={{ width: 40, height: 40, borderRadius: 10, backgrou
                  <div style={{ flex: 1 }}>
                    <div style={{ fontSize: 13, fontFamily: "'Bebas Neue', cursiv
                    <div style={{ fontSize: 10, color: "#666", marginTop: 2 }}>{h
                  </div>
                  <div style={{ textAlign: "right" }}>
                    <div style={{ fontSize: 12, color: "#ef4444", fontFamily: "'B
                    <div style={{ fontSize: 10, color: "#666" }}>~{h.cal} kcal</d
                  </div>
                </div>
              ))}
            </div>
          </div>
        )}

        {(!stats.history || stats.history.length === 0) && (
          <div style={{ background: "rgba(255,255,255,0.02)", border: "1px solid
            <div style={{ fontSize: 36, marginBottom: 12 }}>📊</div>
            <div style={{ fontSize: 13, color: "#666" }}>Aucune séance enregistré
            <div style={{ fontSize: 11, color: "#444", marginTop: 6 }}>Lance le t
          </div>
        )}
      </div>
    </div>
  );
}

// ── SPLASH SCREEN ──────────────────────────────────────────────
function SplashScreen() {
  const [phase, setPhase] = useState(0);

  useEffect(() => {
    const t1 = setTimeout(() => setPhase(1), 400);
    const t2 = setTimeout(() => setPhase(2), 900);
    const t3 = setTimeout(() => setPhase(3), 1400);
    return () => { clearTimeout(t1); clearTimeout(t2); clearTimeout(t3); };
  }, []);

  return (
```

```
<div style={{display:"flex", flexDirection:"column", alignItems:"center", jus
  <style>{`
    @keyframes splashGlow {
      0%,100% { filter: drop-shadow(0 0 12px rgba(239,68,68,0.4)); }
      50% { filter: drop-shadow(0 0 28px rgba(239,68,68,0.9)); }
    }
    @keyframes splashRing {
      from { transform: scale(0.6); opacity: 0; }
      to { transform: scale(1); opacity: 1; }
    }
    @keyframes splashFadeUp {
      from { opacity: 0; transform: translateY(20px); }
      to { opacity: 1; transform: translateY(0); }
    }
    @keyframes splashBar {
      from { width: 0%; }
      to { width: 100%; }
    }
    @keyframes rotateSlow {
      from { transform: rotate(0deg); }
      to { transform: rotate(360deg); }
    }
  `}</style>

  {/* Background decorative rings */}
  <div style={{position: "absolute", width: 340, height: 340, borderRadius: "
  <div style={{position: "absolute", width: 260, height: 260, borderRadius: "
  <div style={{position: "absolute", width: 180, height: 180, borderRadius: "

  {/* Red corner accents */}
  {[[0,0,"tl"],[0,"auto","tr"],["auto",0,"bl"],["auto","auto","br"]].map(([t,
    <div key={id} style={{position: "absolute", top: t !== "auto" ? 24 : "aut
  ))}

  {/* Glove icon */}
  <div style={{fontSize: 72, animation: phase >= 0 ? "splashGlow 2s ease infi

  {/* AUTAM text */}
  <div style={{fontSize: 54, fontFamily: "'Bebas Neue', cursive", color: "#ff

  {/* MUAY THAI text */}
  <div style={{fontSize: 22, fontFamily: "'Bebas Neue', cursive", color: "#ef

  {/* Tagline */}
  <div style={{fontSize: 12, color: "#555", letterSpacing: 3, fontStyle: "ita

  {/* Loading bar */}
```

```
      <div style={{width: 120, height: 2, background: "rgba(255,255,255,0.06)", b
        <div style={{height: "100%", background: "linear-gradient(90deg, #ef4444,
      </div>
    </div>
  );
}

// ── MAIN APP ──────────────────────────────────────────────────
export default function App() {
  const [screen, setScreen] = useState("home");
  const [isPremium, setIsPremium] = useState(false);
  const [dailyAnalyses, setDailyAnalyses] = useState(0);
  const [user, setUser] = useState(null);
  const [authChecked, setAuthChecked] = useState(false);
  const [trialDaysLeft, setTrialDaysLeft] = useState(7);
  const [newBadge, setNewBadge] = useState(null);
  const { shareData, shareProgress, shareBadge, shareDefi, closeShare } = useShar
  const isOnline = useOnlineStatus();

  useEffect(() => {
    const user = supaAuth.getSession();
    if (user) {
      setUser(user);
      const daysLeft = supaAuth.getTrialDaysLeft(user);
      setTrialDaysLeft(daysLeft);
      if (daysLeft > 0) setIsPremium(true);
    }
    setAuthChecked(true);
  }, []);

  const handleAuth = (user) => {
    setUser(user);
    setIsPremium(true);
    setTrialDaysLeft(supaAuth.getTrialDaysLeft(user));
    const stats = badgeStorage.getStats();
    const earned = badgeStorage.checkAndUnlock(stats);
    if(earned.length > 0) setTimeout(() => setNewBadge(earned[0]), 1500);
    setScreen("home");
  };

  const handleSignOut = () => {
    supaAuth.signOut();
    setUser(null);
    setIsPremium(false);
    setScreen("home");
  };
```

```
const goPaywall = () => setScreen("paywall");

if (!authChecked) return <SplashScreen />;

const renderScreen = () => {
  if (!user) return <AuthScreen onAuth={handleAuth} onShowLegal={(tab) => setSc
  switch (screen) {
    case "home": return <HomeScreen setScreen={setScreen} isPremium={isPremium}
    case "timer": return <TimerScreen onSessionComplete={b => setNewBadge(b)} /
    case "techniques": return <TechniquesScreen isPremium={isPremium} goPaywall
    case "programs": return <ProgramsScreen isPremium={isPremium} goPaywall={go
    case "profile": return <ProfileScreen setScreen={setScreen} isPremium={isPr
    case "coach": return <CoachIAScreen isPremium={isPremium} dailyAnalyses={da
    case "paywall": return <PaywallScreen setScreen={setScreen} onActivate={()
    case "stats": return <StatsScreen setScreen={setScreen} />;
    case "reviews": return <ReviewsScreen setScreen={setScreen} user={user} />;
    case "referral": return <ReferralScreen setScreen={setScreen} user={user} i
    case "chat": return <CoachChatScreen setScreen={setScreen} isPremium={isPre
    case "defi": return <DefiDuJourScreen setScreen={setScreen} isPremium={isPr
    case "progression": return <ProgressionScreen setScreen={setScreen} onBadge
    case "badges": return <BadgesScreen setScreen={setScreen} onShare={shareBad
    case "notifications": return <NotificationsScreen setScreen={setScreen} />;
    case "admin": return <AdminScreen setScreen={setScreen} />;
    case "legal-privacy": return <LegalScreen setScreen={setScreen} initialTab=
    case "legal-terms": return <LegalScreen setScreen={setScreen} initialTab="t
    default: return <HomeScreen setScreen={setScreen} isPremium={isPremium} />;
  }
};

return (
  <>
    <style>{GLOBAL_STYLES}</style>
    <div style={{ display:"flex", justifyContent:"center", alignItems:"center",
      <div style={{width:375, height:780, background:D.bg, borderRadius:44, ove
        {/* Status bar */}
        <div style={{ height:44, background:D.bg, display:"flex", alignItems:"c
          <span style={{ fontSize:12, color:D.textPrimary, fontWeight:700, font
          <div style={{ width:120, height:28, background:"#000", borderRadius:2
          <div style={{ display:"flex", gap:6, alignItems:"center" }}>
            {!isOnline && <span style={{ fontSize:10, color:D.gold }}>📵</span>
            {isPremium && <span style={{ fontSize:10, color:D.gold }}>⭐</span>
            <div style={{ display:"flex", gap:3 }}>
              {[1,2,3].map(i => <div key={i} style={{ width:4, height:4, border
            </div>
          </div>
        </div>
```

```
          {/* Screen */}
          <div style={{ height:"calc(100% - 44px)", overflowY:"auto", position:"r
            <OfflineBanner isOnline={isOnline} />
            {renderScreen()}
            {newBadge && <BadgeNotification badge={newBadge} onClose={() => setNe
            {shareData && <ShareCard {...shareData} onClose={closeShare} />}
            {user && ["paywall","legal-privacy","legal-terms","admin","notificat
          </div>
        </div>
      </div>
    </>
  );
}
```